Trabajo Practico

1.Dato informativos

Asignatura:	Teoría de la Programación
Docente:	José Oswaldo Guamán Quinche
Estudiante(s):	Iker Antonio Criollo Valarezo
Curso / Paralelo:	1A
Fecha de entrega:	3 de Noviembre de 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2.Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

EJ-002

En una libreria, construya un algoritmo que lea una temperatura en grados Celsius (real) por teclado. Use una constante simbolica FACTOR_F = 1.8 y otra OFFSET_F = 32. Calcule la temperatura en Fahrenheit con la expresion aritmetica (celsius * FACTOR_F + OFFSET_F) y asigne el resultado a fahrenheit. Muestre por pantalla un mensaje del tipo 'X grados C equivalen a Y grados F'. Identifique claramente el dato de entrada y la informacion de salida. Ademas del calculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos entrada:

Temperatura en Celsius (celsius)

Proceso:

1. Declarar las constantes FACTOR_F = 1.8 y OFFSET_F = 32
2. Declarar las variables reales celsius, farenheit, logaritmo e inicializarlas en 0.0
3. Leer la temperatura en celsius
4. Calcular temperatura en fahrenheit con la formula fahrenheit = celsius * FACTOR_F + OFFSET_F
5. Evaluar si fahrenheit es mayor a 0, calcular el logaritmo base 10, sino asignar 0 a logaritmo y mostrar mensaje de error en el calculo.
6. Mostrar el mensaje con la conversión de grados celsius a fahrenheit junto al valor del logaritmo

Datos salida:

Mensaje con la conversión de celsius a fahrenheit

Valor del logaritmo (logaritmo)

Diagrama de flujo

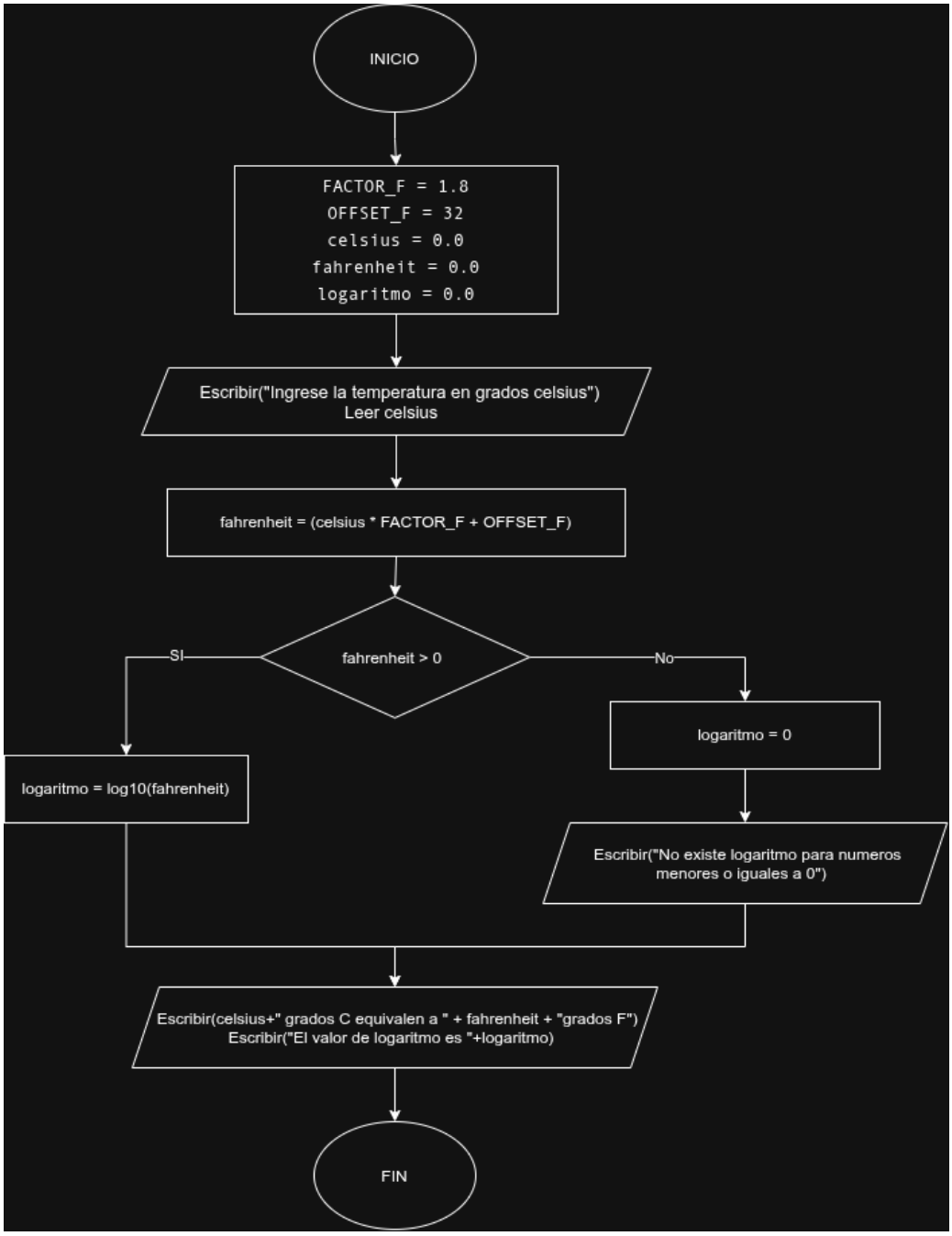


Fig.1. Diagrama de flujo del ejercicio 002

Pseudocódigo

inicio

FACTOR_F = 1,8

OFFSET_F = 32

celsius = 0.0

fahrenheit = 0.0

logaritmo = 0.0

escribir("Ingrese la temperatura en grados Celsius")

leer celsius

fahrenheit = (celsius * FACTOR_F + OFFSET_F)

si fahrenheit > 0 entonces

    logaritmo = log10(fahrenheit)

sino

logaritmo = 0

escribir("No existe logaritmo para números menores o iguales a 0")

FinSi

escribir(celsius + " grados C equivalen a " + fahrenheit + " grados F")

escribir("El valor de logaritmo es " + logaritmo)

fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
celsius	25	25 grados C equivalen a 77 grados F El valor de logaritmo es 1.8865
fahrenheit	77.0	
logaritmo	1.8865	

Prueba 2

Variable	Valor	Resultado
celsius	-40	No existe logaritmo para números menores o iguales a 0 -40 grados C equivalen a -40 grados F El valor de logaritmo es 0.0
fahrenheit	-40.0	
logaritmo	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main()
{
    float FACTOR_F = 1.8;
    int OFFSET_F = 32;

    float celsius = 0.0;
    float fahrenheit = 0.0;
    float logaritmo = 0.0;

    printf("Ingrese la temperatura en grados Celsius\n");
    scanf("%f",&celsius);

    fahrenheit = (celsius*FACTOR_F+OFFSET_F);

    if (fahrenheit > 0 )
    {
        logaritmo = log10(fahrenheit);
    }
    else
    {
        logaritmo = 0.0;
        printf("No existe logaritmo para números menores o iguales a 0\n");
    }

    printf("%f grados C equivalen a %f grados F\n",celsius,fahrenheit);
    printf("El valor de logaritmo es: %f\n",logaritmo);

    return 0;
}

```

Fig.2. Código en C del ejercicio 002

4.7. Salida del programa

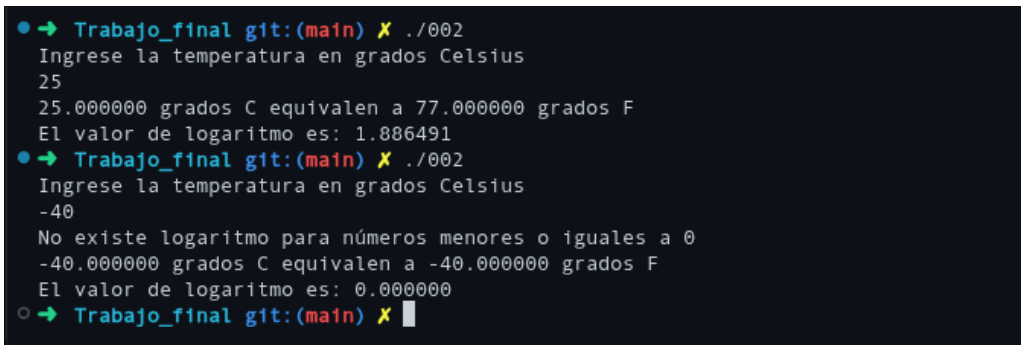


Fig.3. Salida por terminal del código

Conclusiones:

-Utilizar constantes simbólicas como OFFSET_F ayudan a que el código sea fácil de mantener al solo tener que cambiar su valor para que se vea reflejado en todas las partes donde se usa

-Hay ciertas operaciones matemáticas como el logaritmo que solo se pueden aplicar en ciertas condiciones como que el numero sea mayor a 0

EJ-004

En una zapateria, cree un algoritmo que solicite el nombre de una persona (cadena), su año de nacimiento (entero) y el año actual (entero). Use una constante simbólica MAYORIA_EDAD = 18. Calcule la edad con la expresion aritmética (año_actual - año_nacimiento) y asignela a edad. Evalúe la expresión relacional (edad >= MAYORIA_EDAD) y muestre por pantalla 'X tiene Y años y es mayor/menor de edad'. Ademas, calcule el valor mínimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

Análisis del problema

Datos Entrada:

Nombre de la persona (nombre)

El año de nacimiento (anio_nacimiento)

El año actual (anio_actual)

Proceso:

1. Declarar la constante simbólica MAYORIA_EDAD y asignarle el valor de 18
2. Declarar las variables nombre, anio_nacimiento, anio_actual, edad y minimo
3. Recibir los datos de nombre, anio_nacimiento y anio_actual mediante la entrada por teclado
4. Validar que anio_actual y anio_nacimiento sean mayores que 0 y que anio_actual sea mayor o igual que anio_nacimiento
5. Si alguna condición no se cumple mostrar un mensaje de error y finalizar el programa
6. Calcular la edad de la persona con la fórmula (edad = anio_actual - anio_nacimiento)
7. Verificar si la persona es mayor de edad mediante la expresión relacional (edad >= MAYORIA_EDAD)
8. Mostrar por terminal el mensaje “nombre tiene edad años y es mayor/menor de edad”
9. Asignar a minimo el valor de anio_nacimiento

10. Determinar el valor mínimo de todas las variables usando de forma secuencial condicionales comparando entre sí los valores de cada una de las variables numéricas (anio_nacimiento, anio_actual, edad, MAYORIA_EDAD) y asignar ese resultado a la variable minimo
11. Mostrar por terminal el valor mínimo de todas las variables utilizadas

Datos Salida:

Mensaje con el nombre de la persona, su edad y si es mayor o menor de edad

El valor mínimo de las variables utilizadas

Mensaje de error de datos ingresados no válidos en caso de no cumplirse las condiciones

Diagrama de flujo

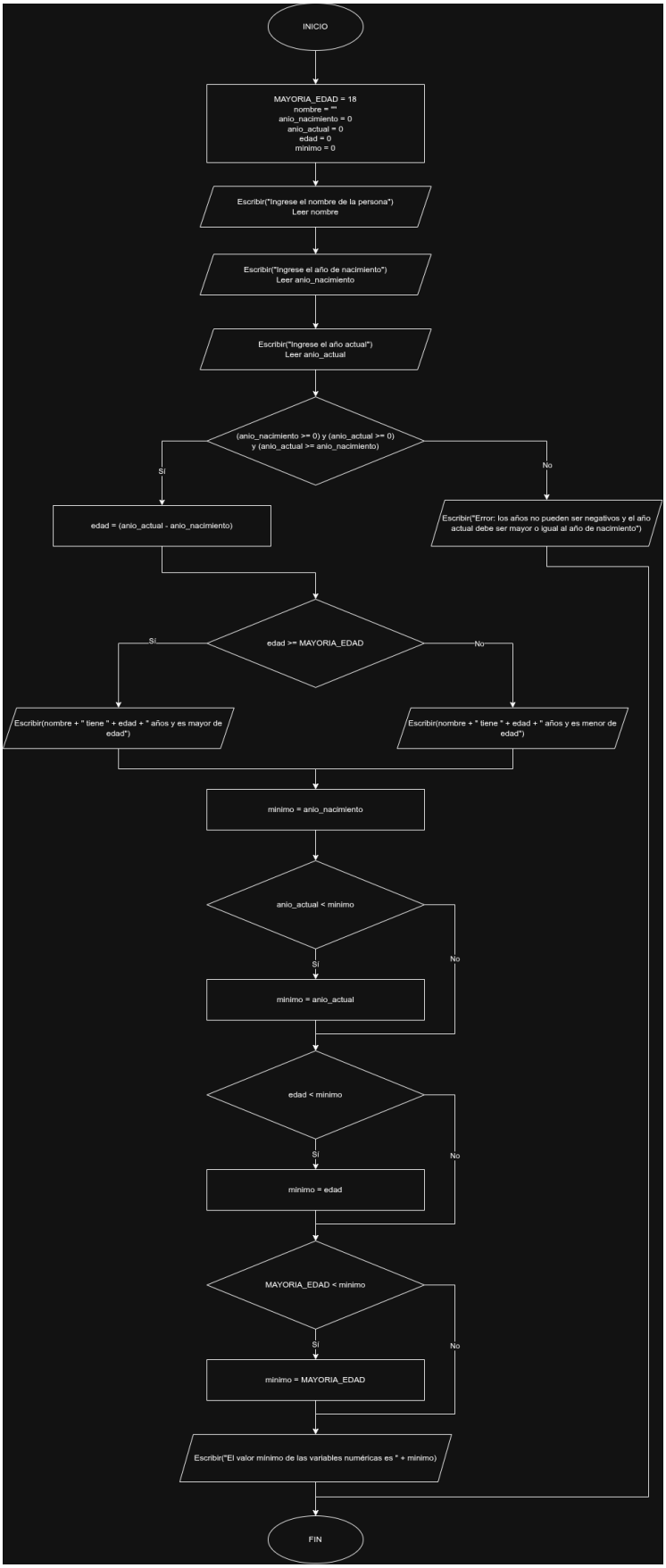


Fig.4. Diagrama de flujo del ejercicio 004

Pseudocódigo

```
Inicio
        MAYORIA_EDAD = 18

        nombre = ""
        anio_nacimiento = 0
        anio_actual = 0
        edad = 0
        minimo = 0

        escribir("Ingrese el nombre de la persona")
        leer nombre

        escribir("Ingrese el año de nacimiento")
        leer anio_nacimiento

        escribir("Ingrese el año actual")
        leer anio_actual

        si (anio_nacimiento >= 0) y (anio_actual >= 0) y (anio_actual >= anio_nacimiento) entonces

           edad = (anio_actual - anio_nacimiento)

           si edad >= MAYORIA_EDAD entonces
              escribir(nombre + " tiene " + edad + " años y es mayor de edad")
           sino
              escribir(nombre + " tiene " + edad + " años y es menor de edad")
           FinSi
```

```
minimo = anio_nacimiento


si anio_actual < minimo entonces
        minimo = anio_actual
FinSi
si  edad < minimo entonces
        minimo = edad
FinSi
si MAYORIA_EDAD < minimo entonces
        minimo = MAYORIA_EDAD
FinSi
   escribir("El valor mínimo de las variables numéricas es " + minimo)


sino
     escribir("Error: los años no pueden ser negativos y el año actual debe ser mayor o igual al año de nacimiento")
FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
nombre	"Ana"	Ana tiene 26 años y es mayor de edad El valor mínimo de las variables numéricas es 18
anio_nacimiento	2000	
anio_actual	2026	
edad	26	
minimo	18	

Prueba 2

Variable	Valor	Resultado
nombre	"Luis"	Luis tiene 16 años y es menor de edad El valor mínimo de las variables numéricas es 16
anio_nacimiento	2010	
anio_actual	2026	
edad	16	
minimo	16	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <string.h>

int main()
{
    int MAYORIA_EDAD  =18;
    char nombre[100];
    int anio_nacimiento = 0;
    int anio_actual = 0;
    int edad = 0;
    int minimo = 0;

    printf("Ingrese el nombre de la persona\n");
    fgets(nombre,sizeof(nombre),stdin);
    nombre[strcspn(nombre,"\n")] = '\0';
    printf("Ingrese el año de nacimiento\n");
    scanf("%d",&anio_nacimiento);
    printf("Ingrese el año actual\n");
    scanf("%d",&anio_actual);

    if (anio_nacimiento >= 0 && anio_actual >= 0 && anio_actual >= anio_nacimiento)
    {
        edad = anio_actual-anio_nacimiento;

        if (edad >= MAYORIA_EDAD)
        {
            printf("%s tiene %d años y es mayor de edad\n",nombre,edad);
        }
        else
        {
            printf("%s tiene %d años y es menor de edad\n",nombre,edad);
        }

        minimo = anio_nacimiento;

        if (anio_actual < minimo)
        {
            minimo = anio_actual;
        }
        if (edad < minimo)
        {
            minimo = edad;
        }
        if (MAYORIA_EDAD < minimo)
        {
            minimo = MAYORIA_EDAD;
        }


        printf("El valor minimo de las variables numéricas es: %d\n",minimo);


    }
    else
    {
        printf("Error: los años no pueden ser negativos y el año actual debe ser mayor o igual al de nacimiento\n");
    }




    return 0;
}

```

Fig.5. Código en C del ejercicio 004

4.7. Salida del programa

```
Ingrese el nombre de la persona
Ana
Ingrese el año de nacimiento
2000
Ingrese el año actual
2026
Ana tiene 26 años y es mayor de edad
El valor mínimo de las variables numéricas es: 18
➜ Trabajo_final git:(main) ✗
```

Fig.6. Salida en terminal del programa

```
Ingrese el nombre de la persona
Luis
Ingrese el año de nacimiento
2010
Ingrese el año actual
2026
Luis tiene 16 años y es menor de edad
El valor mínimo de las variables numéricas es: 16
➜ Trabajo_final git:(main) ✗
```

Fig.7. Salida en terminal del programa

Conclusiones:

-El método para buscar el valor mínimo es un ejemplo de una estructura secuencial

-Usar constantes simbólicas como MAYORIA_EDAD en conjunto de expresiones relacionales ayuda a ver de forma mas clara que es lo que se esta comparando y con que fin

EJ-005

En una cooperativa, declare una variable entera a y una constante literal entera 7. Lea a por teclado. Calcule el residuo de la división (a % 7) y asigne el resultado a residuo. Evalúe la expresión lógica (residuo == 0 AND a > 0). Muestre por pantalla el mensaje 'Es múltiplo de 7' o 'No es múltiplo de 7'. Indique el tipo de dato simple de cada variable usada. Ademas, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

Datos Entrada:

Numero entero (a)

Variable Usadas:

Nombre	Clase	Tipo de Dato
a	Variable	Entero
residuo	Variable	Entero
7	Constante Literal	Entero

Proceso:

1. Declarar las variables a, residuo como tipo entero e inicializarlas en 0
2. Recibir y almacenar el valor de a
3. Realizar la operación (a % 7) y asignar el resultado a residuo
4. Evaluar la expresión (residuo == 0 AND a > 0) y mostrar "Es múltiplo de 7" si se cumple la expresión o "No es múltiplo de 7" si no se cumple
5. Evaluar si residuo es par o impar usando la expresión relacional (residuo % 2 == 0) y mostrar por terminal

Datos Salida:

Mensajes informativos durante la ejecución

Mensaje indicando si el numero es o no múltiplo de 7

Mensaje indicando si residuo es par o impar

Diagrama de flujo

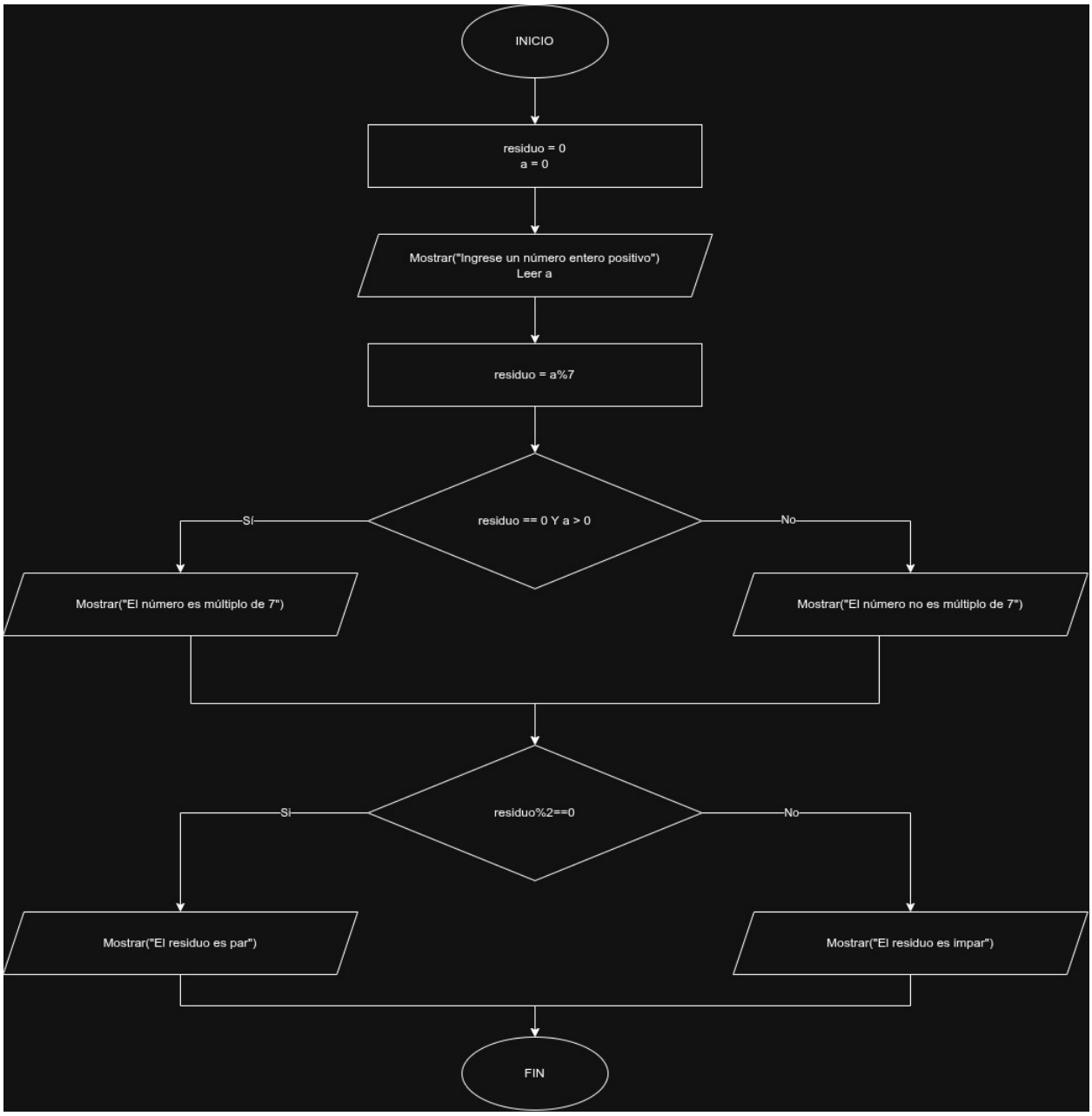


Fig.8. Diagrama de flujo del ejercicio 005

Pseudocódigo

```
Inicio

    residuo =
    a = 0

    Mostrar("Ingrese un número entero positivo")
    Leer a

    residuo = a%7
    si residuo == 0 Y a > 0 Entonces
        Mostrar("El número es múltiplo de 7")
    sino
        Mostrar("El número no es múltiplo de 7")
    FinSi

    si residuo%2==0 Entonces
        Mostrar("El residuo es par")
    sino
        Mostrar("El residuo es impar")
    FinSi

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
a	14	El número es múltiplo de 7 El residuo es par
residuo	0	

Prueba 2

Variable	Valor	Resultado
a	10	El número no es múltiplo de 7 El residuo es impar
residuo	3	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{

    int residuo = 0;
    int a = 0;

    printf("Ingrese un número entero positivo\n");
    scanf("%d",&a);

    residuo = a%7;
    if (residuo == 0 && a > 0 )
    {
        printf("El número es múltiplo de 7\n");
    }
    else
    {
        printf("El número no es múltiplo de 7\n");
    }

    if (residuo%2 == 0)
    {
        printf("El residuo es par\n");
    }
    else
    {
        printf("El residuo es impar\n");
    }



    return 0;
}

```

Fig.9. Codigo en C del ejercicio 005

4.7. Salida del programa

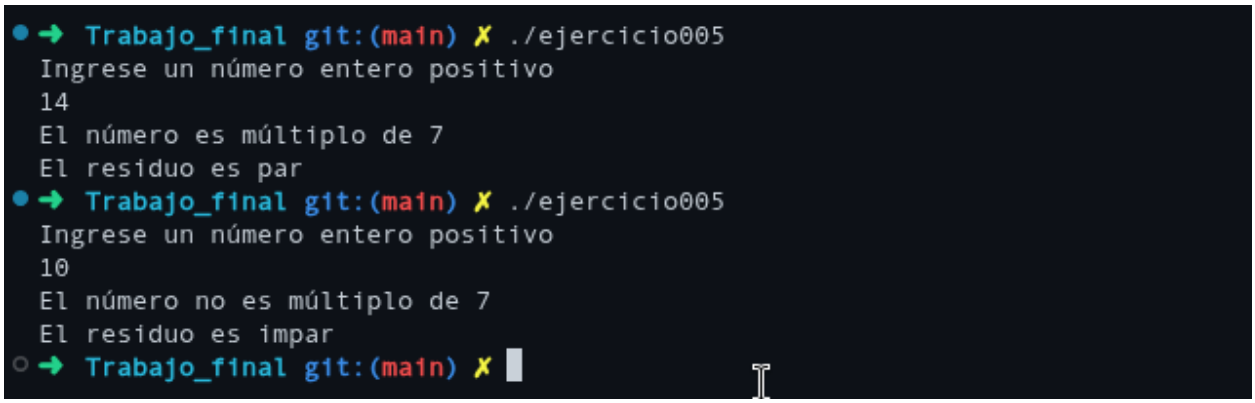


Fig.10. Salida por terminal del ejercicio

Conclusiones:

-En las expresiones lógicas se pueden utilizar elementos como el AND para hacer que una condición se vuelva mas estricta

- Se puede utilizar la operación del modulo como una herramienta para clasificar números según cierta condición como si son pares o impares

EJ-006

En una panaderia, construya un algoritmo que lea el nombre de un estudiante (cadena) y tres notas (reales entre 0 y 10). Use una constante simbólica NOTA_MIN = 7.0. Calcule el promedio con la expresion aritmetica ((nota1 + nota2 + nota3) / 3) y asignelo a promedio. Evalúe la expresion relacional (promedio >= NOTA_MIN). Muestre por pantalla el nombre del estudiante, su promedio y si aprobo o reprobo. Ademas, calcule el coseno del angulo resultado (si aplica). Agregue un mensaje indicando el tipo de operacion realizada.

Análisis del problema

Datos Entrada:

Nombre del estudiante (nombre)

Nota 1 (nota1)

Nota 2 (nota2)

Nota 3 (nota3)

Proceso:

1. Declarar la constante simbólica NOTA_MIN y asignarle el valor de 7.0
2. Declarar las variables nombre como cadena de caracteres, nota1, nota2, nota3 y promedio como reales
3. Registrar el nombre y las 3 notas
4. Verificar si las 3 notas se encuentran dentro del rango de 0 a 10 incluidos cada uno
5. Si alguna nota no cumple mostrar mensaje de error y terminar el programa
6. Calcular el promedio con la formula (nota1+nota2+nota3)/3 y asignar el resultado a promedio

7. Verificar la expresión relacional (promedio >= NOTA_MIN)
8. Mostrar nombre, promedio y si aprobó o no dependiendo del resultado
9. Calcular y mostrar el valor del coseno del promedio mediante la función cos(promedio)
10. Mostrar el tipo de operación realizada

Datos Salida:

Mensaje que dice nombre del estudiante, promedio y si aprobó o no

Mensaje con el valor del coseno del promedio

Mensaje con el tipo de operación realizada

Mensaje de error si las notas no están dentro del rango

Diagrama de flujo

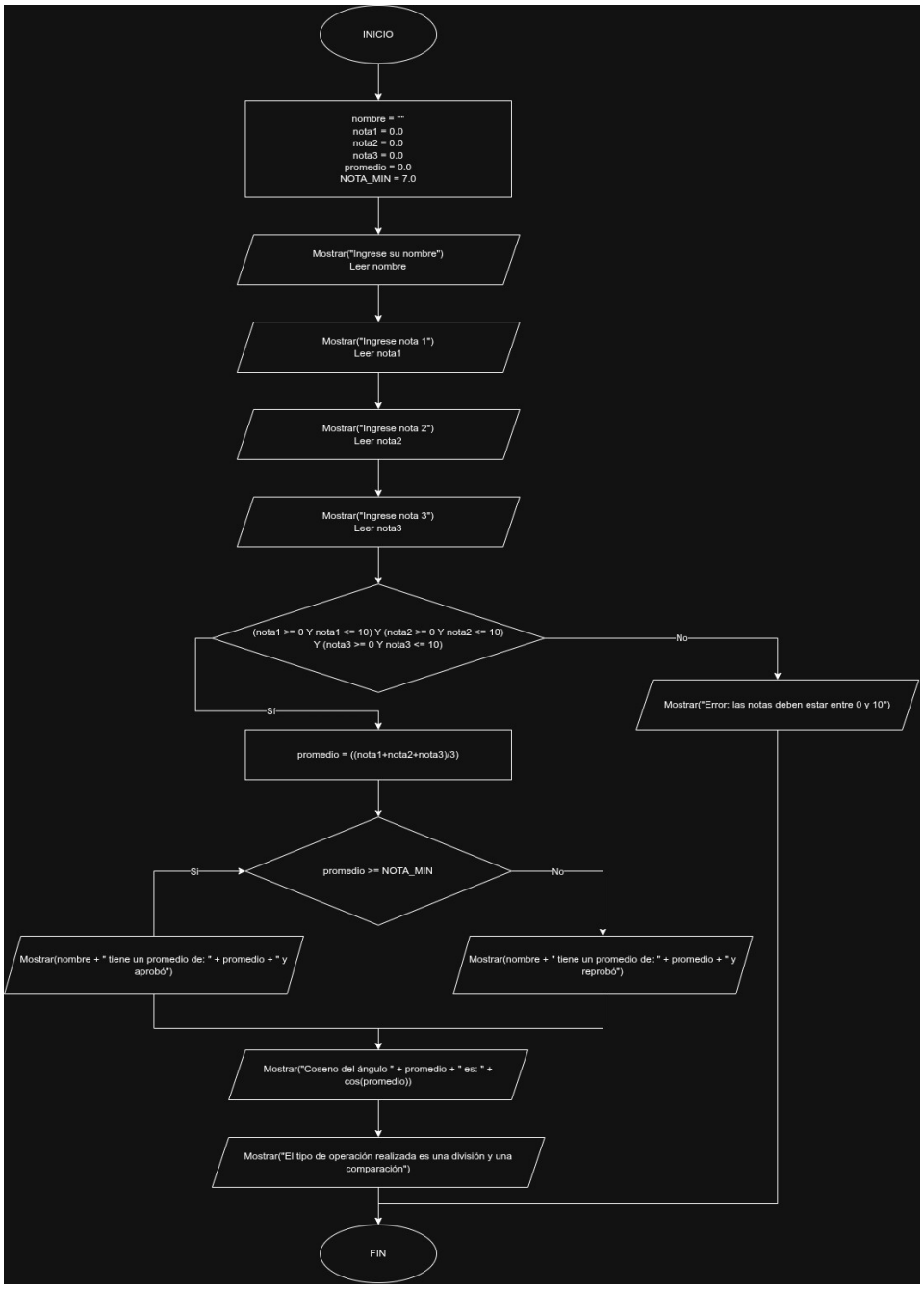


Fig.11. Diagrama de flujo del ejercicio 006

Pseudocódigo

```
Inicio


        nombre = ""
        nota1 = 0.0
nota2 = 0.0
nota3 = 0.0
promedio = 0.0
        NOTA_MIN  = 7.0


        Mostrar ("Ingrese su nombre")
        Leer nombre
        Mostrar ("Ingrese nota 1")
        Leer  nota1
        Mostrar ("Ingrese nota 2")
        Leer  nota2
        Mostrar ("Ingrese nota 3")
        Leer nota3
        si (nota1 >= 0 Y nota1 <= 10) Y (nota2 >= 0 Y nota2 <= 10) Y (nota3 >= 0 Y nota3 <= 10) entonces
        promedio = ((nota1+nota2+nota3)/3)


        si promedio>=NOTA_MIN Entonces
                Mostrar (nombre + " tiene un promedio de: " + promedio + " y aprobó")
        sino
                Mostrar (nombre + " tiene un promedio de: " + promedio + " y reprobó")
        FinSi



        Mostrar ("Coseno del ángulo " + promedio +" es: "+cos(promedio))
```

```
            Mostrar("El tipo de operación realizada es promedio y coseno")
        sino
            mostrar("Error: las notas deben estar entre 0 y 10")
        FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
nombre	"María"	María tiene un promedio de: 9.0 y aprobó Coseno del ángulo 9.0 es: -0.9111 El tipo de operación realizada es promedio y coseno
nota1	8.0	
nota2	9.0	
nota3	10.0	
promedio	9.0	

Prueba 2

Variable	Valor	Resultado
nombre	"Pedro"	Pedro tiene un promedio de: 5.0 y reprobó Coseno del ángulo 5.0 es: 0.2837 El tipo de operación realizada es promedio y coseno
nota1	5.0	
nota2	6.0	
nota3	4.0	
promedio	5.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <string.h>
#include <math.h>

int main()
{
    char nombre[100];
    float nota1 = 0.0;
    float nota2 = 0.0;
    float nota3 = 0.0;
    float promedio = 0.0;
    float NOTA_MIN = 7.0;

    printf("Ingrese su nombre\n");
    fgets(nombre,sizeof(nombre),stdin);
    nombre[strcspn(nombre,"\n")] = '\0';

    printf("Ingrese nota 1\n");
    scanf("%f",&nota1);
    printf("Ingrese nota 2\n");
    scanf("%f",&nota2);
    printf("Ingrese nota 3\n");
    scanf("%f",&nota3);

    if (nota1 >= 0 && nota1 <=10 && nota2>= 0 && nota2<=10 && nota3>=0 && nota3 <=10)
    {
        promedio = (nota1+nota2+nota3)/3;

        if (promedio >= NOTA_MIN)
        {
            printf("%s tiene un promedio de: %.2f y aprobó\n",nombre,promedio);
        }
        else
        {
            printf("%s tiene un promedio de: %.2f y reprobó\n",nombre,promedio);
        }

        printf("Coseno del ángulo %.2f es: %f\n", promedio, cos(promedio));
        printf("El tipo de operación realizada es promedio y coseno\n");

    }

    else
    {
        printf("Error: las notas deben estar entre 0 y 10\n");
    }



    return 0;
}

```

Fig.12.Código en C del ejercicio 006

4.7. Salida del programa

```
Trabajo_final git:(main) ✗ ./006
Ingrese su nombre
Maria
Ingrese nota 1
8
Ingrese nota 2
9
Ingrese nota 3
10
Maria tiene un promedio de: 9.00 y aprobó
Coseno del ángulo 9.00 es: -0.911130
El tipo de operación realizada es promedio y coseno
```

Fig.13. Salida por terminal del programa

```
Trabajo_final git:(main) ✗ ./006
Ingrese su nombre
Pedro
Ingrese nota 1
5
Ingrese nota 2
6
Ingrese nota 3
4
Pedro tiene un promedio de: 5.00 y reprobó
Coseno del ángulo 5.00 es: 0.283662
El tipo de operación realizada es promedio y coseno
```

Fig.14. Salida por terminal del programa

Conclusiones:

-Validar al inicio que los valores ingresados (notas) estén dentro del rango permitido (1 a 10) ayuda a prevenir resultados erróneos

-Con este ejercicio aprendí que cada tipo de dato puede tener una forma propia de manejar sus valores como en el caso de nombre que tuve que usar fgets para guardar un valor por terminal

EJ-007

En un hospital, diseñe un algoritmo que lea una cantidad de segundos (entero). Use constantes literales 60 y 3600. Calcule las horas con (segundos / 3600), los minutos con ((segundos % 3600) / 60) y los segundos restantes con (segundos % 60). Asigne cada resultado a variables enteras h, m, s. Muestre por pantalla el formato 'H horas, M minutos y S segundos'. Identifique el tipo de dato de cada variable. Ademas, calcule el inverso multiplicativo del resultado si no es cero. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos Entrada:

Cantidad de segundos (segundos)

Variables Usadas:

Nombre	Clase	Tipo de dato
segundos	Variable	Entero
h	Variable	Entero
m	Variable	Entero
s	Variable	Entero
inv_h	Variable	Real
inv_m	Variable	Real
inv_s	Variable	Real
3600	Constante literal	Entero
60	Constante literal	Entero

Proceso:

1. Declarar las variables segundos, h, m, s como enteras y inv_h, inv_m e inv_s como reales
2. Recibir por teclado el valor de segundos
3. Verificar que segundos >= 0
4. Si no se cumple mostrar mensaje de Error y terminar el programa
5. Calcular h con la formular h = segundos/3600
6. Calcular m con la formula m = (segundos%3600)/60
7. Calcular s con la formula s = segundos%60
8. Mostrar por terminal el mensaje “H horas, M minutos y S segundos”
9. Verificar h > 0, si se cumple calcular inv_h = 1.0/h y mostrar el inverso de h, si no mostrar que el inverso no esta definido
10. Verificar m > 0, si se cumple calcular inv_m = 1.0/m y mostrar el inverso de m, si no mostrar que el inverso no esta definido

1. Verificar s > 0, si se cumple calcular inv_s = 1.0/s y mostrar el inverso de s, si no mostrar que el inverso no esta definido
2. Mostrar el valor de todas las variables auxiliares

Datos Salida:

Mensaje de error si los segundos son menores a 0

Mensaje con el formato “H horas, M minutos y S segundos”

Mensajes con el inverso multiplicativo de h, m y s

Mensaje indicando que los inversos no están definidos cuando el valor es 0

Mensaje con los valores de todas las variables auxiliares

Diagrama de flujo

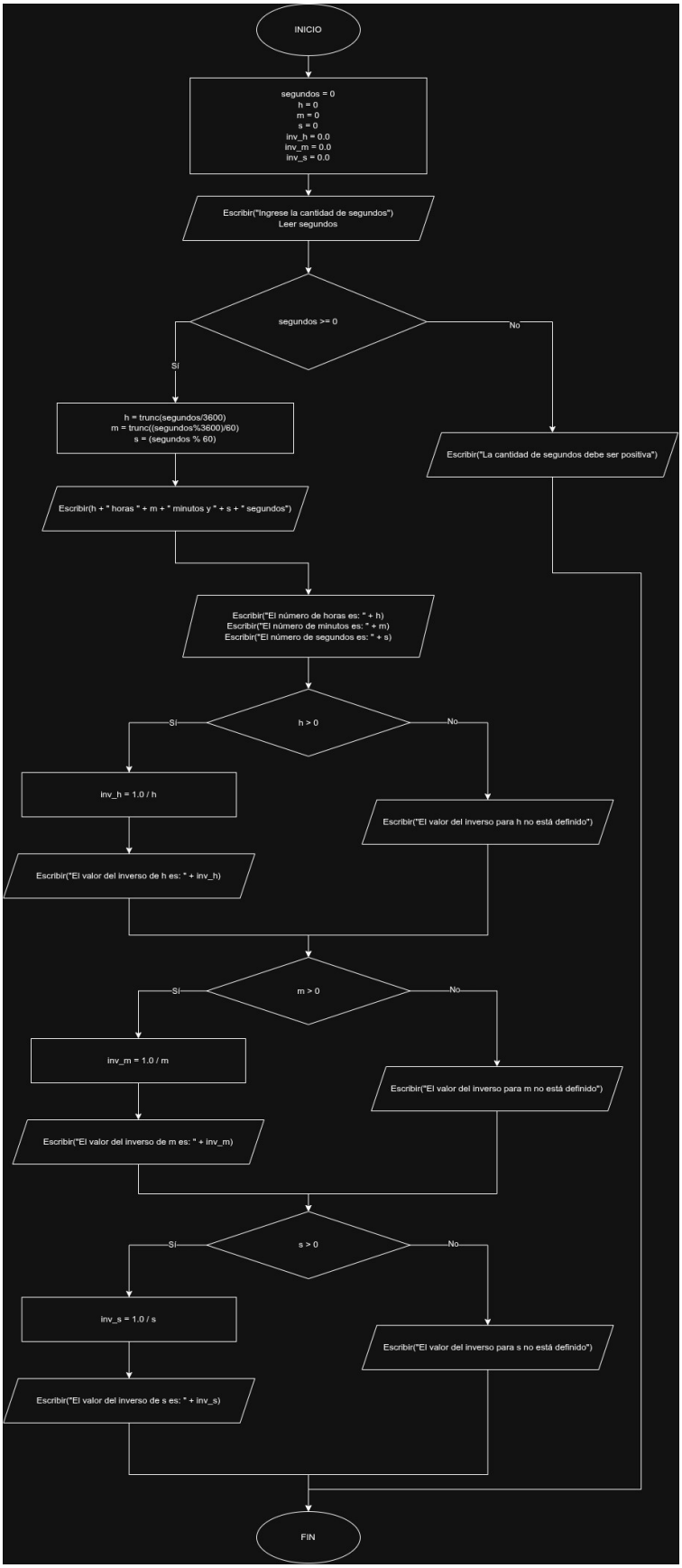


Fig.15. Diagrama de flujo del ejercicio 007

Pseudocódigo

Inicio

segundos = 0

h = 0

m = 0

s = 0

inv_h = 0.0

inv_m = 0.0

inv_s = 0.0

escribir(" Ingrese la cantidad de segundos ")

Leer segundos

si segundos >= 0 entonces

h =trunc(segundos/3600)

m = trunc((segundos%3600)/60)

s = (segundos % 60)

escribir(h+" horas "+m+" minutos y "+s+ " segundos")

escribir("El número de horas es: "+h)

escribir("El número de minutos es: "+m)

escribir("El número de segundos es: "+s)

si h > 0 entonces

inv_h = 1.0/h

escribir("El valor del inverso de h es: " + inv_h)

sino

escribir("El valor del inverso para h no está definido")

FinSi

```
        si m > 0 entonces
                inv_m = 1.0/m
                escribir("El valor del inverso de m es: " + inv_m)
        sino
                escribir("El valor del inverso para m no está definido")
        FinSi


        si s > 0 entonces
                inv_s = 1.0/s
                escribir("El valor del inverso de s es: " + inv_s)
        sino
                escribir("El valor del inverso para s no está definido")
        FinSi



sino
        escribir("La cantidad de segundos debe ser positiva")
FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
segundos	3661	1 horas 1 minutos y 1 segundos El número de horas es: 1 El número de minutos es: 1 El número de segundos es: 1 El valor del inverso de h es: 1.0 El valor del inverso de m es: 1.0 El valor del inverso de s es: 1.0
h	1	
m	1	
s	1	
inv_h	1.0	
inv_m	1.0	
inv_s	1.0	

Prueba 2

Variable	Valor	Resultado
segundos	3600	1 horas 0 minutos y 0 segundos El número de horas es: 1 El número de minutos es: 0 El número de segundos es: 0 El valor del inverso de h es: 1.0 El valor del inverso para m no está definido El valor del inverso para s no está definido
h	1	
m	0	
s	0	
inv_h	1.0	
inv_m	0.0	
inv_s	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{
    int segundos = 0;
    int h = 0;
    int m = 0;
    int s = 0;

    float inv_h = 0.0;
    float inv_m = 0.0;
    float inv_s = 0.0;

    printf("Ingrese la cantidad de segundos\n");
    scanf("%d",&segundos);

    if (segundos >= 0)
    {
        h = segundos/3600;
        m = ((segundos%3600)/60);
        s = (segundos % 60);
        printf("%d horas, %d minutos y %d segundos\n",h,m,s);
        printf("El número de horas es: %d\n", h);
        printf("El número de minutos es: %d\n", m);
        printf("El número de segundos es: %d\n", s);

        if (h > 0)
        {
            inv_h = 1.0/h;
            printf("El valor del inverso de h es: %.2f\n",inv_h);
        }
        else
        {
            printf("El valor del inverso para h no está definido\n");
        }

        if (m > 0)
        {
            inv_m = 1.0/m;
            printf("El valor del inverso de m es: %.2f\n",inv_m);
        }
        else
        {
            printf("El valor del inverso para m no está definido\n");
        }
        if (s > 0)
        {
            inv_s = 1.0/s;
            printf("El valor del inverso de s es: %.2f\n",inv_s);
        }
        else
        {
            printf("El valor del inverso para s no está definido\n");
        }

    }
    else
    {
        printf("La cantidad de segundos debe ser positiva\n");
    }


    return 0;
}

```

Fig.16. Código en C del ejercicio 007

4.7. Salida del programa

```
Ingrese la cantidad de segundos
3661
1 horas, 1 minutos y 1 segundos
El número de horas es: 1
El número de minutos es: 1
El número de segundos es: 1
El valor del inverso de h es: 1.00
El valor del inverso de m es: 1.00
El valor del inverso de s es: 1.00
```

Fig.17. Salida por terminal del programa

```
Ingrese la cantidad de segundos
3600
1 horas, 0 minutos y 0 segundos
El número de horas es: 1
El número de minutos es: 0
El número de segundos es: 0
El valor del inverso de h es: 1.00
El valor del inverso para m no está definido
El valor del inverso para s no está definido
```

Fig.18. Salida por terminal del programa

Conclusiones:

-Aprendí como con operadores como la división o el modulo, se pueden separar elementos específicos de un dato

-Con este ejercicio me di cuenta de que siempre hay que tener cuidado, con los requisitos para que una operación se pueda realizar, como por ejemplo para sacar el inverso de un numero este no puede ser 0

EJ-008

En una farmacia, cree un algoritmo que lea la base (real) y la altura (real) de un rectángulo. Use la constante literal 2 en el perímetro. Calcule área = base * altura y perímetro = 2 * (base + altura) usando expresiones aritméticas. Evalúe la expresión lógica (base > 0 AND altura > 0). Asigne los resultados a variables reales. Muestre por pantalla el área y el perímetro. Ademas del calculo principal, determine el doble del resultado. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

Datos Entrada:

Base del rectángulo (base)

Altura del rectángulo (altura)

Proceso:

1. Declarar las variables reales base, altura, area, perimetro, doble_area y doble_perimetro e inicializarlas en 0.0.
2. Recibir la base del rectángulo por teclado.
3. Recibir la altura del rectángulo por teclado.
4. Verificar la expresión lógica (base > 0 AND altura > 0).
5. Si no se cumple mostrar por terminal un mensaje de error y terminar el programa.
6. Calcular el área con la fórmula area = base * altura.
7. Calcular el perímetro usando la constante literal 2 con la fórmula perimetro = 2 * (base + altura).
8. Determinar el doble del área con la fórmula doble_area = area * 2.
9. Determinar el doble del perímetro con la fórmula doble_perimetro = perimetro * 2.
10. Mostrar por terminal el área del rectángulo.
11. Mostrar por terminal el perímetro del rectángulo.
12. Mostrar por terminal el doble del área.
13. Mostrar por terminal el doble del perímetro.
14. Mostrar por terminal un mensaje de confirmación de que la operación finalizó correctamente.

Datos Salida:

Mensaje con el área del rectángulo

Mensaje con el perímetro del rectángulo

Mensaje con el doble del área

Mensaje con el doble del perímetro

Mensaje de confirmación al finalizar la operación

Mensaje de error si la base o la altura no son mayores que 0

Diagrama de flujo

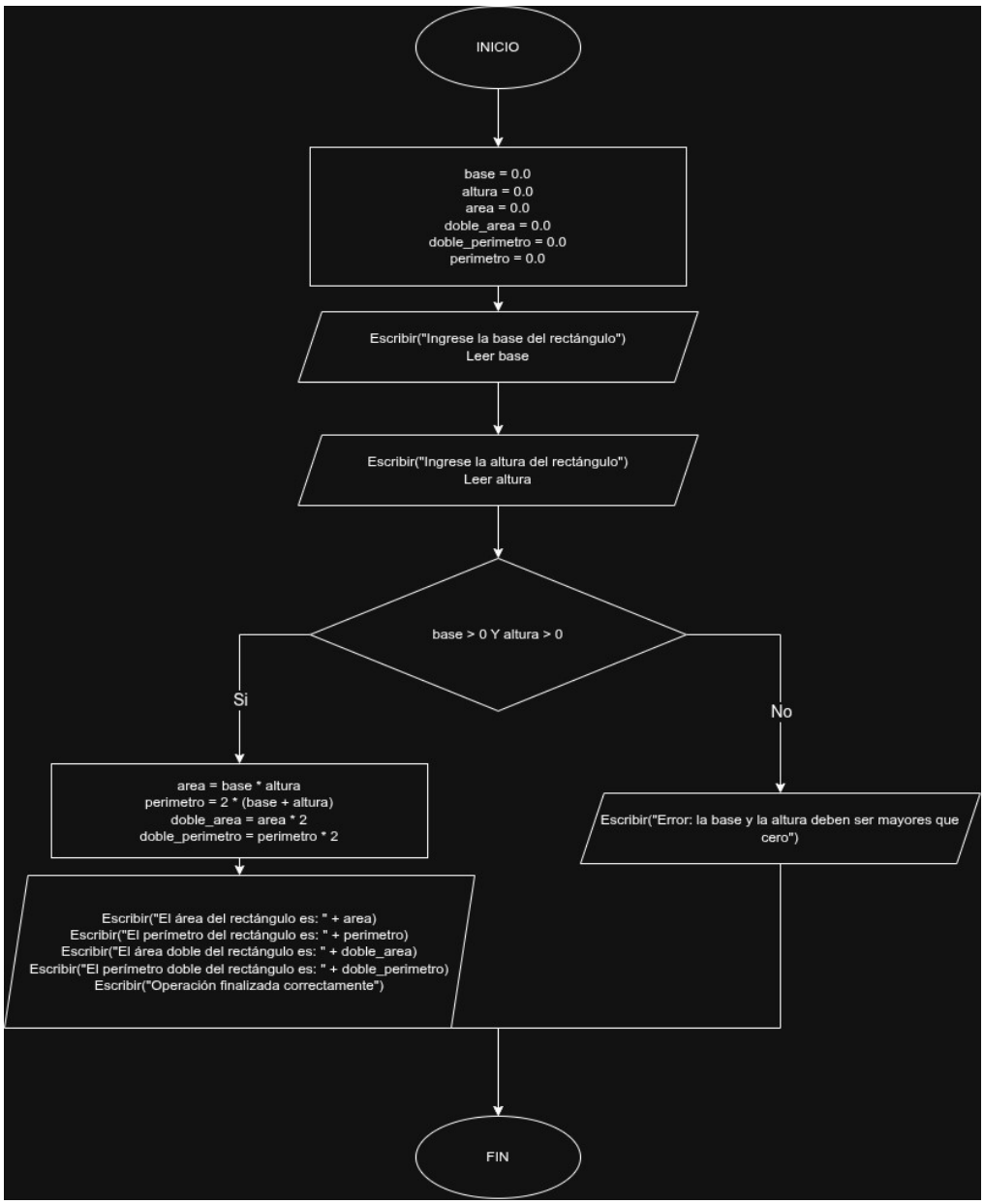


Fig.19. Diagrama de flujo del ejercicio 008

Pseudocódigo

Inicio

base = 0.0

altura = 0.0

area = 0.0

doble_area = 0.0

```
doble_perimetro =  0.0
perimetro = 0.0

escribir("Ingrese la base del rectángulo")
leer base

escribir("Ingrese la altura del rectángulo")
leer altura

si  base > 0 Y altura > 0 entonces

area = base * altura
perimetro = 2 * (base + altura)
doble_area = area * 2
            doble_perimetro = perimetro *2

   escribir("El área del rectángulo es: " + area)
   escribir("El perímetro del rectángulo es: " + perimetro)
   escribir("El área doble del rectángulo es: " + doble_area)
               escribir("El perímetro doble del rectángulo es: " + doble_perimetro)
   escribir("Operación finalizada correctamente")

sino
   escribir("Error: la base y la altura deben ser mayores que cero")
FinSi

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
base	5.0	El área del rectángulo es: 15.0 El perímetro del rectángulo es: 16.0 El área doble del rectángulo es: 30.0 El perímetro doble del rectángulo es: 32.0 Operación finalizada correctamente
altura	3.0	
area	15.0	
perimetro	16.0	
doble_area	30.0	
doble_perimetro	32.0	

Prueba 2

Variable	Valor	Resultado
base	2.5	El área del rectángulo es: 10.0 El perímetro del rectángulo es: 13.0 El área doble del rectángulo es: 20.0 El perímetro doble del rectángulo es: 26.0 Operación finalizada correctamente
altura	4.0	
area	10.0	
perimetro	13.0	
doble_area	20.0	
doble_perimetro	26.0	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{

    float base = 0.0;
    float altura = 0.0;
    float area = 0.0;
    float doble_area = 0.0;
    float doble_perimetro = 0.0;
    float perimetro = 0.0;


    printf("Ingrese la base del rectángulo\n");
    scanf("%f",&base);
    printf("Ingrese la altura del rectángulo\n");
    scanf("%f",&altura);

    if (base > 0 && altura > 0)
    {
        area = base * altura;
        perimetro = 2*(base+altura);
        doble_area = area*2;
        doble_perimetro = perimetro*2;

        printf("El área del rectángulo es: %f\n",area);
        printf("El perímetro del rectángulo es: %f\n", perimetro);
        printf("El área doble del rectángulo es: %f\n", doble_area);
        printf("El perímetro doble del rectángulo es: %f\n", doble_perimetro);
        printf("Operación finalizada correctamente\n");
    }
    else
    {
        printf("Error: la base y la altura deben ser mayores que cero\n");
    }


    return 0;
}

```

Fig.20. Código en C del ejercicio 008

4.7. Salida del programa

```
Ingrese la base del rectángulo
5
Ingrese la altura del rectángulo
3
El área del rectángulo es: 15.000000
El perímetro del rectángulo es: 16.000000
El área doble del rectángulo es: 30.000000
El perímetro doble del rectángulo es: 32.000000
Operación finalizada correctamente
```

Fig.21. Salida por terminal del programa

```
Ingrese la base del rectángulo
2.5
Ingrese la altura del rectángulo
4
El área del rectángulo es: 10.000000
El perímetro del rectángulo es: 13.000000
El área doble del rectángulo es: 20.000000
El perímetro doble del rectángulo es: 26.000000
Operación finalizada correctamente
```

Fig.22. Salida por terminal del programa

Conclusiones:

-Descubrí la diferencia entre una constante literal y una simbólica, mientras que la simbólica adquiere un nombre como PI, las literales se escriben directamente como en este caso el 2

-Aprendí que con el condicional del If-Else se puede validar un dato antes de operarlo

EJ-009

En una escuela, construya un algoritmo que lea el precio de un producto (real) y la cantidad (entero). Use una constante simbólica IVA = 0.15. Calcule el subtotal con (precio * cantidad), el monto del IVA con (subtotal * IVA) y el total a pagar con (subtotal + iva). Evalúe la expresion relacional (total > 0). Muestre por pantalla un desglose completo. Ademas del calculo principal, multiplique el resultado por 1.5 como bono. Agregue validaciones para que no haya resultados negativos.

Análisis del problema

Datos Entrada:

Precio del producto (precio)

Cantidad de productos (cantidad)

Proceso:

1. Declarar la constante simbólica IVA = 0.15.
2. Declarar las variables reales precio, subtotal, monto_iva, total y bono, la variable entera cantidad y la variable booleana valido

3. Recibir el precio del producto por teclado.
4. Recibir la cantidad de productos por teclado.
5. Verificar que el precio y la cantidad no sean negativos.
6. Si no se cumple mostrar por terminal un mensaje de error y terminar el programa.
7. Calcular el subtotal con la fórmula subtotal = precio * cantidad.
8. Calcular el monto del IVA con la fórmula monto_iva = subtotal * IVA.
9. Calcular el total a pagar con la fórmula total = subtotal + monto_iva.
10. Evaluar la expresión relacional total > 0
11. Si se cumple determinar el bono con (bono = total * 1.5), si no se cumple bono = 0
12. Mostrar por terminal el precio unitario y la cantidad de productos.
13. Mostrar por terminal el subtotal, el monto del IVA y el total a pagar.
14. Mostrar por terminal el resultado de la expresión relacional (total > 0).
15. Mostrar por terminal el total multiplicado por 1.5.

Datos Salida:

Mensaje con el precio unitario y la cantidad de productos

Mensaje con el subtotal, el monto del IVA y el total a pagar

Mensaje con el resultado de la expresión relacional (total > 0)

Mensaje con el total multiplicado por 1.5

Mensaje de error si el precio o la cantidad son negativos

Diagrama de flujo

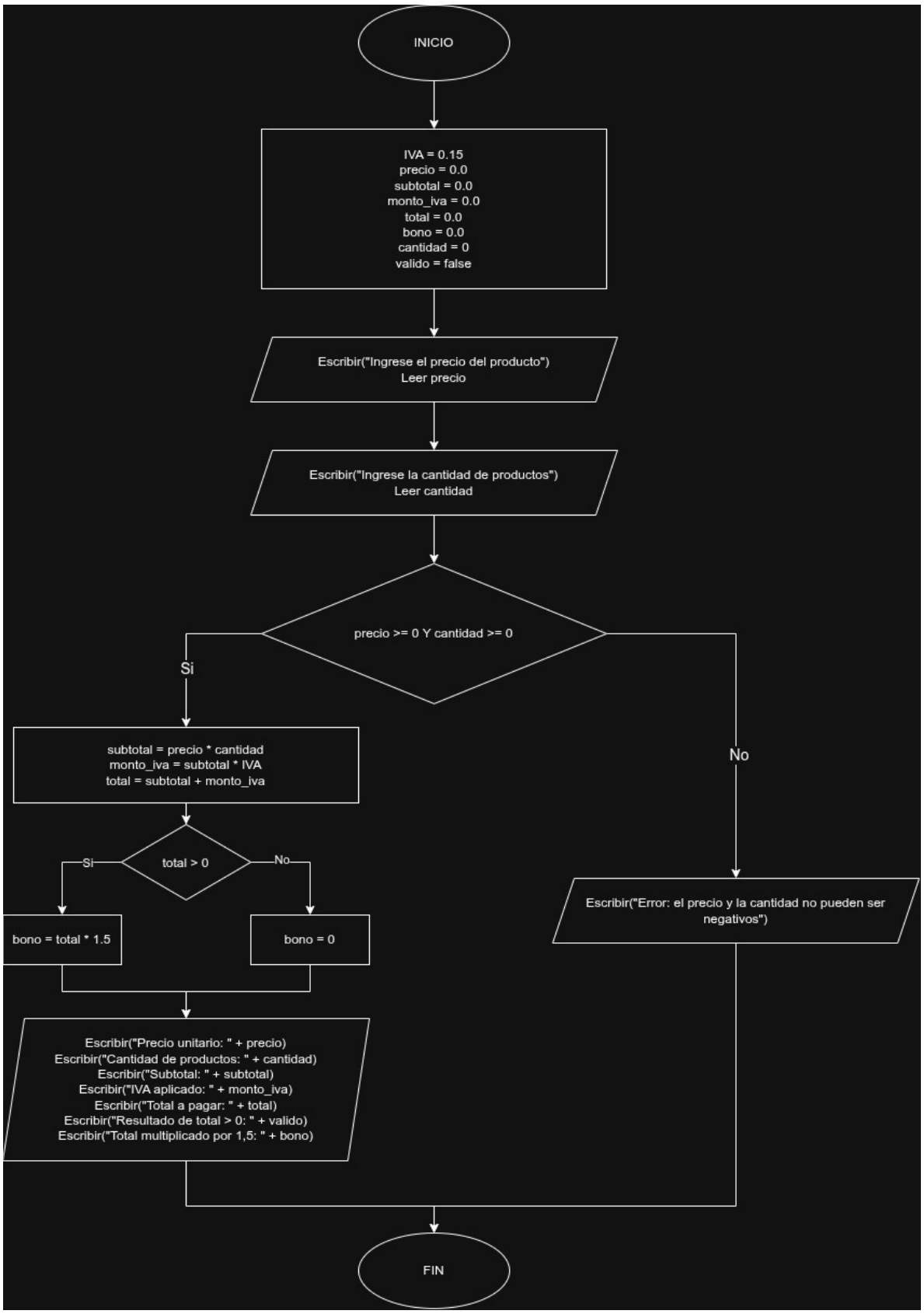


Fig.23. Diagrama de flujo del ejercicio 009

Pseudocódigo

```
Inicio

    IVA = 0.15
    precio = 0.0
    subtotal = 0.0
    monto_iva = 0.0
    total = 0.0
    bono  = 0.0
    cantidad = 0
    valido = false

    escribir("Ingrese el precio del producto")
    leer precio

    escribir("Ingrese la cantidad de productos")
    leer cantidad

    si precio >= 0 Y cantidad >= 0 entonces

        subtotal = precio * cantidad
        monto_iva = subtotal * IVA
        total = subtotal + monto_iva
        si total > 0 entonces
            bono = total * 1,5
        sino
            bono = 0

        escribir("Precio unitario: " + precio)
```

```
            escribir("Cantidad de productos: " + cantidad)
            escribir("Subtotal: " + subtotal)
            escribir("IVA aplicado: " + monto_iva)
            escribir("Total a pagar: " + total)
            escribir("Resultado de total > 0: " + valido)
            escribir("Total multiplicado por 1,5: " + bono)

        sino
            escribir("Error: el precio y la cantidad no pueden ser negativos")
        FinSi

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
precio	10.0	Precio unitario: 10.0 Cantidad de productos: 3 Subtotal: 30.0 IVA aplicado: 4.5 Total a pagar: 34.5 Resultado de total > 0: true Total multiplicado por 1,5: 51.75
cantidad	3	
subtotal	30.0	
monto_iva	4.5	
total	34.5	
bono	51.75	
valido	true	

Prueba 2

Variable	Valor	Resultado
precio	1.2	Precio unitario: 1.2 Cantidad de productos: 5 Subtotal: 6.0 IVA aplicado: 0.90 Total a pagar: 6.90 Resultado de total > 0: false Total multiplicado por 1,5: 10.35
cantidad	5	
subtotal	6.0	
monto_iva	0.90	
total	6.90	
bono	10.35	
valido	false	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    float IVA = 0.15;
    float precio = 0.0;
    float subtotal = 0.0;
    float monto_iva = 0.0;
    float total = 0.0;
    float bono = 0.0;
    int cantidad = 0;
    bool valido = false;

    printf("Ingrese el precio del producto\n");
    scanf("%f",&precio);
    printf("Ingrese la cantidad de productos\n");
    scanf("%d",&cantidad);

    if (precio >= 0 && cantidad >= 0)
    {
        subtotal = precio * cantidad;
        monto_iva = subtotal * IVA;
        total = subtotal + monto_iva;
        if (total > 0 )
        {
            bono = total * 1.5;
        }
        else
        {
            bono = 0;
        }


        printf("Precio unitario: %.2f\n", precio);
        printf("Cantidad de productos: %d\n", cantidad);
        printf("Subtotal: %.2f\n", subtotal);
        printf("IVA aplicado: %.2f\n", monto_iva);
        printf("Total a pagar: %.2f\n", total);
        printf("Resultado de total > 0: %d\n", valido);
        printf("Total multiplicado por 1.5: %.2f\n", bono);
    }
    else
    {
        printf("Error: el precio y la cantidad no pueden ser negativos\n");
    }

    return 0;
}

```

Fig.24. Código en C del ejercicio 009

4.7. Salida del programa

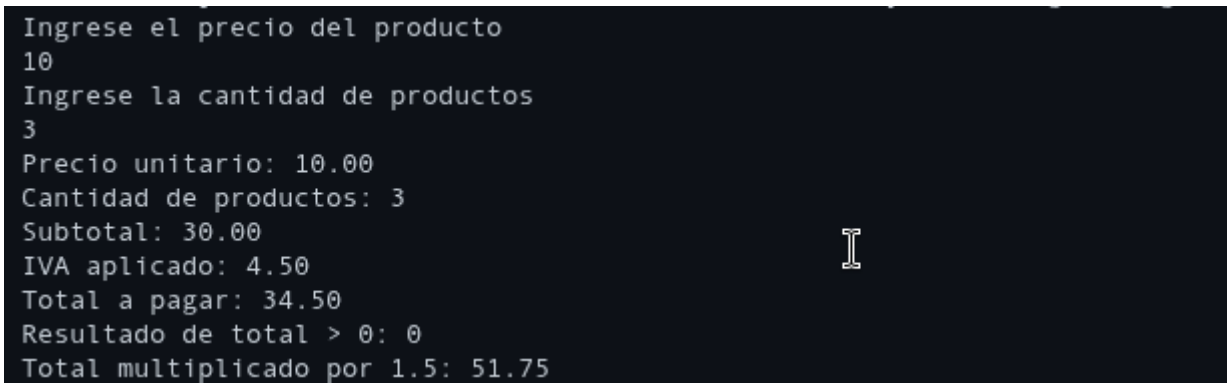


Fig.25. Salida por terminal del ejercicio

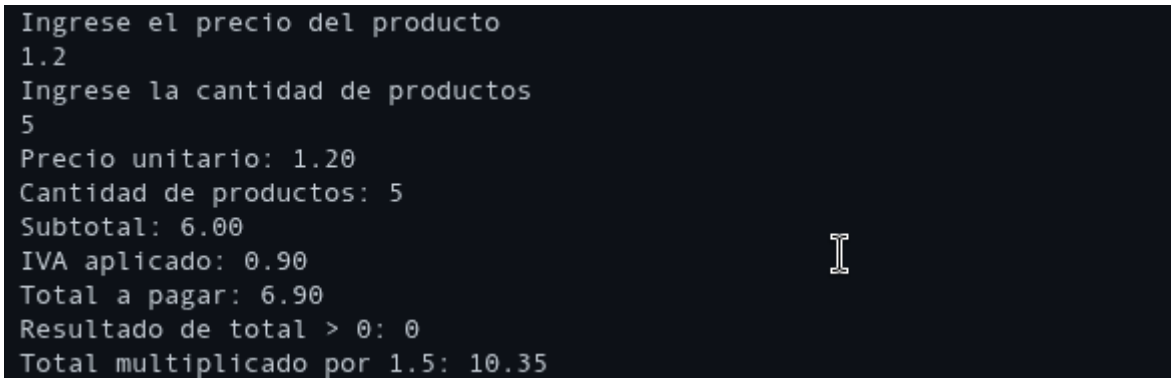


Fig.26. Salido por terminal del ejercicio

Conclusiones:

-Este ejercicio muestra como un operación puede depender completamente del resultado de una anterior trabajando de forma secuencial

-En c los boleanos representan su estado con el 1 y el 0

EJ-011

En un cine, construya un algoritmo que lea un carácter por teclado y lo almacene en la variable y. Use constantes literales 'A' y 'Z'. Evalúe la expresión lógica (y >= 'A' AND y <= 'Z'). Asigne el resultado a una variable booleana es_mayúscula. Muestre por pantalla un mensaje indicando si el carácter es o no mayúscula. Identifique el tipo de dato de y y de es_mayúscula. Ademas del calculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operación realizada.

Análisis del problema

Datos Entrada:

Carácter ingresado por teclado (y)

Variables usadas:

Nombre	Clase	Tipo de dato
y	Variable	Carácter
es_mayuscula	Variable	Booleano
resultado	Variable	Entero
producto	Variable	Real
PI	Constante simbólica	Real
'A'	Constante literal	Carácter
'Z'	Constante literal	Carácter

Proceso:

1. Declarar la constante simbólica PI = 3.1416.
2. Declarar las constantes literales 'A' y 'Z'.
3. Declarar la variable y como carácter, es_mayuscula como booleana, resultado como entero y la variable producto como real
4. Recibir y por teclado
5. Evaluar la expresión lógica (y >= 'A' AND y <= 'Z') y asignar el resultado a es_mayuscula.
6. Verificar si es_mayuscula es verdadera.
7. Si es verdadera mostrar por terminal el mensaje “El carácter ingresado es mayúscula” y asignar 1 a resultado
8. Si es falsa mostrar por terminal el mensaje “El carácter ingresado no es mayúscula” y asignar 0 a resultado
9. Calcular el producto del resultado por PI con la fórmula producto = resultado * PI.
10. Mostrar por terminal el producto del resultado por PI.
11. Mostrar por terminal un mensaje indicando que la operación realizada es una multiplicación.

Datos Salida:

Mensaje indicando si el carácter ingresado es mayúscula o no

Mensaje con el producto del resultado por PI

Mensaje indicando que la operación realizada es una multiplicación

Diagrama de flujo

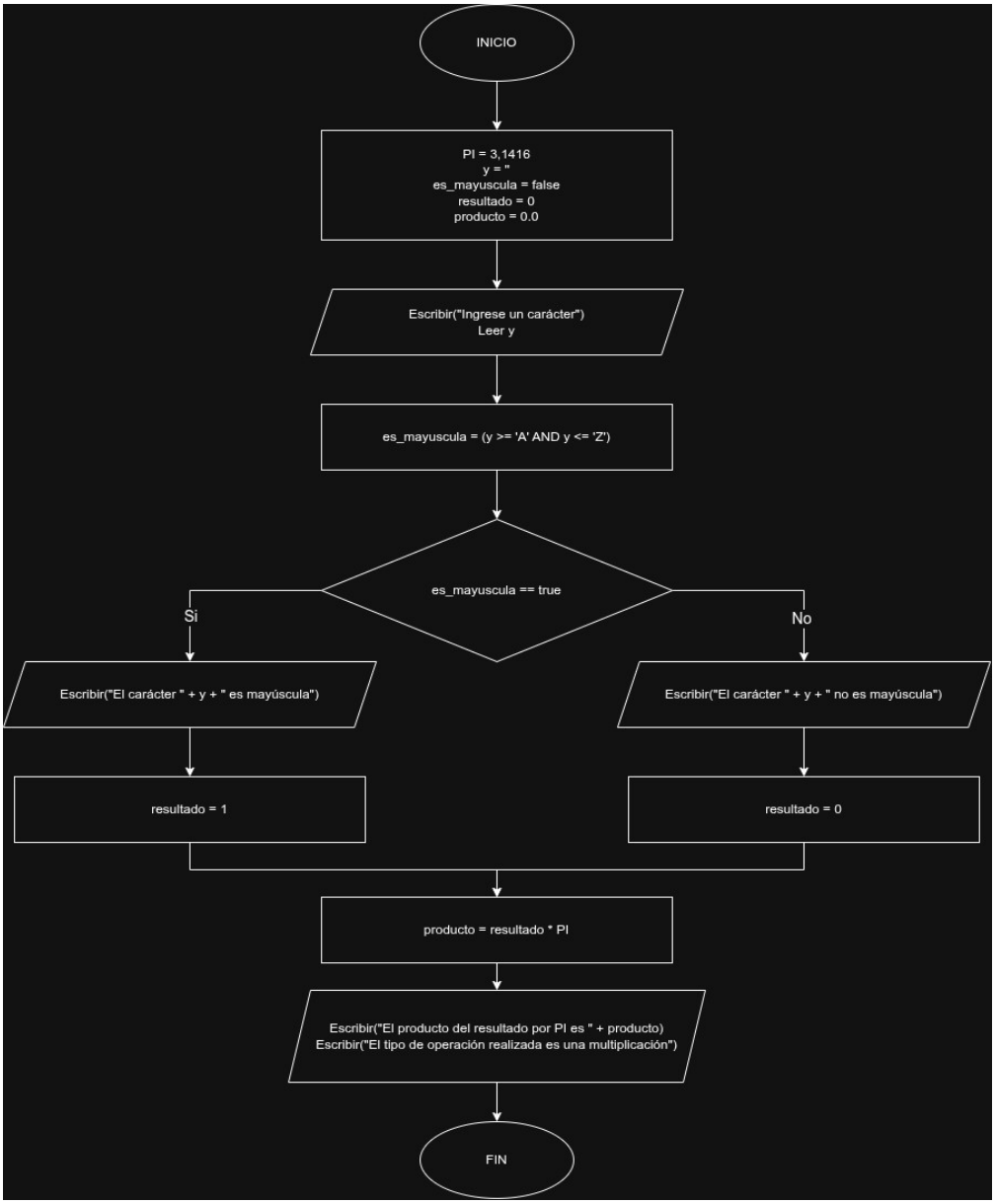


Fig.27. Diagrama de flujo del ejercicio 011

Pseudocódigo

Inicio

PI = 3,1416

y = ''

es_mayuscula = false

resultado = 0

```
producto = 0.0

escribir("Ingrese un carácter")
leer y

es_mayuscula = (y >= 'A' AND y <= 'Z')

si es_mayuscula == true entonces
    escribir("El carácter " + y + " es mayúscula")
    resultado = 1
sino
    escribir("El carácter " + y + " no es mayúscula")
    resultado = 0
FinSi

producto = resultado * PI

escribir("El producto del resultado por PI es " + producto)
escribir("El tipo de operación realizada es una multiplicación")

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
y	'G'	El carácter G es mayúscula El producto del resultado por PI es 3.1416 El tipo de operación realizada es una multiplicación
es_mayuscula	true	
resultado	1	
producto	3.1416	

Prueba 2

Variable	Valor	Resultado
y	'g'	El carácter g no es mayúscula El producto del resultado por PI es 0.0 El tipo de operación realizada es una multiplicación
es_mayuscula	false	
resultado	0	
producto	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    float PI = 3.1416;
    char y = ' ';
    bool es_mayuscula = false;
    int resultado = 0;
    float producto = 0.0;

    printf("Ingrese un carácter\n");
    scanf(" %c",&y);

    es_mayuscula = (y >= 'A' && y <= 'Z');

    if (es_mayuscula == true)
    {
        printf("El carácter %c es mayúscula\n", y);
        resultado = 1;
    }
    else
    {
        printf("El carácter %c no es mayúscula\n", y);
        resultado = 0;
    }

    producto = resultado * PI;

    printf("El producto del resultado por PI es %f\n", producto);
    printf("El tipo de operación realizada es una multiplicación\n");

    return 0;
}
```

Fig.28. Código en C del ejercicio 011

4.7. Salida del programa

```
Trabajo_final git:(main) ✗ ./ejercicio011
Ingrese un carácter
G
El carácter G es mayúscula
El producto del resultado por PI es 3.141600
El tipo de operación realizada es una multiplicación
Trabajo_final git:(main) ✗ ./ejercicio011
Ingrese un carácter
g
El carácter g no es mayúscula
El producto del resultado por PI es 0.000000
El tipo de operación realizada es una multiplicación
Trabajo_final git:(main) ✗
```

Fig.29. Salida por terminal del programa

Conclusión:

-Con este ejercicio aprendí que en C elementos como los caracteres son interpretados como números permitiendo realizar una mayor cantidad de operaciones con ellos

-En C se puede usar una estructura condicional para comprar elementos como un numero y un carácter debido a por como interpreta C los caracteres

EJ-012

En una juguetera, diseñe un algoritmo que lea un carácter por teclado en la variable n. Evalúe las expresiones lógicas: (n == 'a' OR n == 'e' OR n == 'i' OR n == 'o' OR n == 'u'). Asigne el resultado booleano a es_vocal_minuscula. Muestre por pantalla el resultado. Justifique el uso del operador OR en lugar de AND. Ademas, calcule el cuadrado del resultado principal. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos Entrada:

Carácter ingresado por teclado (n)

Proceso:

1. Declarar la variable n como tipo carácter, la variable es_vocal_minuscula como tipo booleana, la variable resultado y cuadrado como tipo entero
2. Recibir un carácter n por teclado
3. Evaluar la expresión lógica (n == 'a' OR n == 'e' OR n == 'i' OR n == 'o' OR n == 'u') y asignar el resultado a es_vocal_minuscula
4. Verificar si es_vocal_minuscula es verdadera
5. Si es verdadera, asignar 1 a resultado y mostrar por terminal el mensaje “El carácter ingresado es una vocal minúscula”
6. Si es falsa, asignar 0 a resultado y mostrar por terminal el mensaje “El carácter ingresado no es una vocal minúscula”
7. Calcular el cuadrado del resultado principal con la fórmula cuadrado = resultado * resultado
8. Mostrar por terminal el valor de n

9. Mostrar por terminal el valor de es_vocal_minuscula
10. Mostrar por terminal el valor de resultado
11. Mostrar por terminal el cuadrado del resultado principal

Datos Salida:

Mensaje indicando si el carácter ingresado es una vocal minúscula o no

Mensaje con el valor de n

Mensaje con el valor de es_vocal_minuscula

Mensaje con el valor de resultado

Mensaje con el cuadrado del resultado principal

Se usa el operador OR porque basta con que el carácter coincida con una de las vocales para que la condición sea verdadera, mientras que con AND tendría que coincidir con todas al mismo tiempo.

Diagrama de flujo

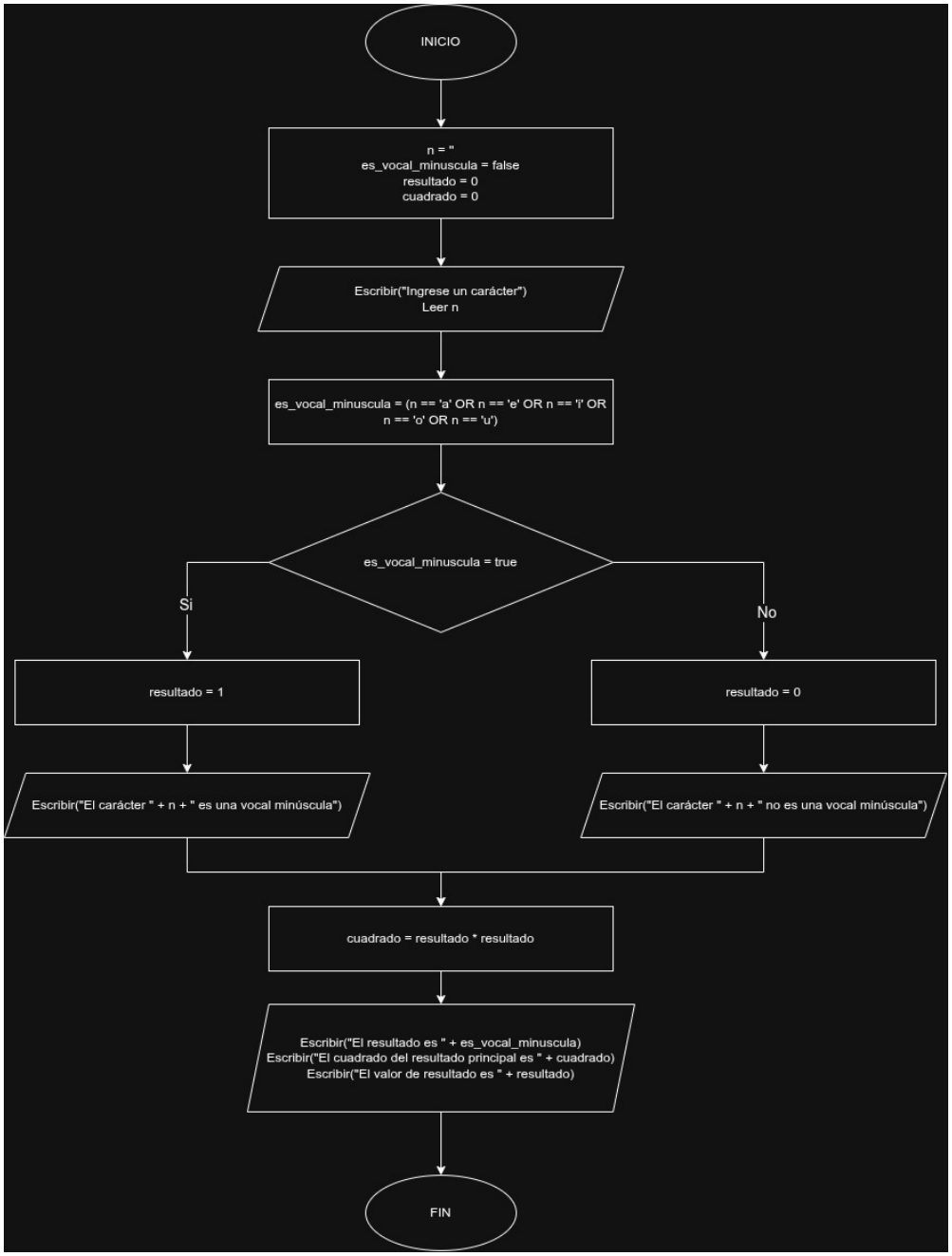


Fig.30. Diagrama de flujo del ejercicio 012

Pseudocódigo

Inicio

n = ''

es_vocal_minuscula = false

resultado = 0

cuadrado = 0

```
    escribir("Ingrese un carácter")
    leer n

    es_vocal_minuscula = (n == 'a' OR n == 'e' OR n == 'i' OR n == 'o' OR n == 'u')

    si es_vocal_minuscula = true entonces
       resultado = 1
       escribir("El carácter " + n + " es una vocal minúscula")
    sino
       resultado = 0
       escribir("El carácter " + n + " no es una vocal minúscula")
    FinSi

    cuadrado = resultado * resultado
    escribir("El resultado es " + es_vocal_minuscula)
    escribir("El cuadrado del resultado principal es " + cuadrado)
    escribir("El valor de resultado es " + resultado)
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
n	'a'	El carácter a es una vocal minúscula El resultado es true El cuadrado del resultado principal es 1 El valor de resultado es 1
es_vocal_minuscula	true	
resultado	1	
cuadrado	1	

Prueba 2

Variable	Valor	Resultado
n	'b'	El carácter b no es una vocal minúscula El resultado es false El cuadrado del resultado principal es 0 El valor de resultado es 0
es_vocal_minuscula	false	
resultado	0	
cuadrado	0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    char n = ' ';
    bool es_vocal_minuscula = 0;
    int resultado = 0;
    int cuadrado = 0;

    printf("Ingrese un carácter\n");
    scanf(" %c",&n);

    es_vocal_minuscula = (n == 'a' || n == 'e' || n == 'i' || n == 'o' || n == 'u');

    if (es_vocal_minuscula == true)
    {
        resultado = 1;
        printf("El carácter %c es una vocal minúscula\n", n);
    }
    else
    {
        resultado = 0;
        printf("El carácter %c no es una vocal minúscula\n", n);
    }

    cuadrado = resultado * resultado;

    printf("El resultado es %d\n", es_vocal_minuscula);
    printf("El cuadrado del resultado principal es %d\n", cuadrado);
    printf("El valor de resultado es %d\n", resultado);

    return 0;
}

```

Fig.31. Código en C del ejercicio 012

4.7. Salida del programa

```
➜ Trabajo_final git:(main) ✗ ./ejercicio012
Ingrese un carácter
a
El carácter a es una vocal minúscula
El resultado es 1
El cuadrado del resultado principal es 1
El valor de resultado es 1
➜ Trabajo_final git:(main) ✗ ./ejercicio012
Ingrese un carácter
b
El carácter b no es una vocal minúscula
El resultado es 0
El cuadrado del resultado principal es 0
El valor de resultado es 0
➜ Trabajo_final git:(main) ✗
```

Fig.32. Salida en terminal del programa

Conclusiones:

-Aprendí que el operador || permite realizar varias comparaciones a la vez y basta con que una sola sea verdadera para que se ejecute la acción

-En este problema descubrí que una variable booleana puede servir como un resumen del resultado de una comparación

EJ-017

En una libreria, construya un algoritmo que lea un monto en euros (real). Use una constante simbolica TASA_CAMBIO = 1.08. Calcule el equivalente en dolares con la expresion aritmetica (euros * TASA_CAMBIO) y asignelo a dolares. Evalúe la expresion relacional (euros > 0). Muestre por pantalla 'X euros equivalen a Y dolares'. Ademas del calculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos Entrada:

Monto en euros (euros)

Proceso:

1. Declarar la constante simbólica TASA_CAMBIO = 1.08
2. Declarar las variables euros, dolares y logaritmo e inicializarlas en 0.
3. Recibir el monto en euros por teclado
4. Verificar que se cumpla la expresión relacional (euros > 0)
5. Si no se cumple mostrar por terminal un mensaje de error y terminar el programa
6. Calcular el equivalente en dólares con la fórmula dolares = euros * TASA_CAMBIO
7. Calcular el logaritmo en base 10 del resultado con la fórmula logaritmo = log10(dolares)
8. Mostrar por terminal el mensaje “X euros equivalen a Y dólares”
9. Mostrar por terminal el valor de logaritmo base 10 del resultado
10. Mostrar por terminal el valor de euros
11. Mostrar por terminal el valor de TASA_CAMBIO

12. Mostrar por terminal el valor de dolares
13. Mostrar por terminal el valor de logaritmo

Datos Salida:

Mensaje con la equivalencia de euros a dólares

Mensaje con el valor de euros

Mensaje con el valor de TASA_CAMBIO

Mensaje con el valor de dolares

Mensaje con el valor de logaritmo

Mensaje de error si el monto en euros no es mayor que 0

Diagrama de flujo

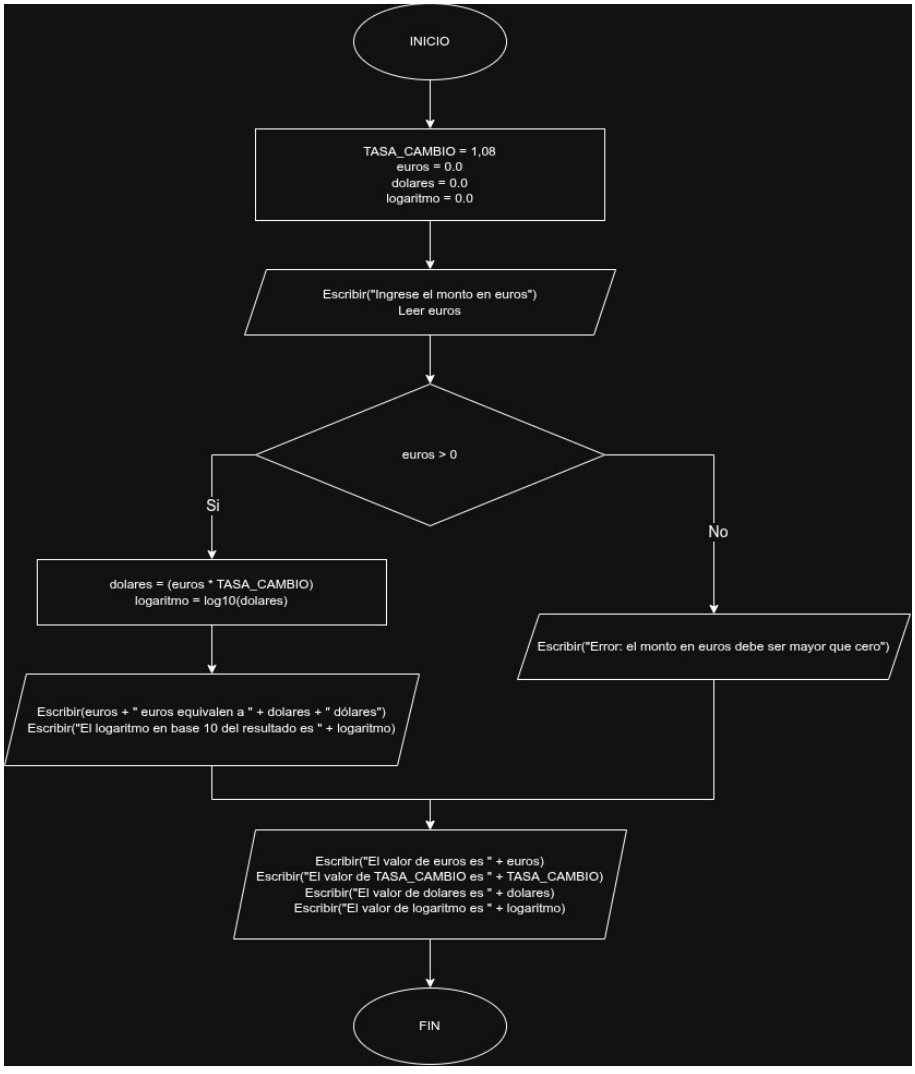


Fig.33. Diagrama de flujo del ejercicio 017

Pseudocódigo

```
Inicio

    TASA_CAMBIO = 1,08

    euros = 0.0
    dolares = 0.0
    logaritmo = 0.0

    escribir("Ingrese el monto en euros")
    leer euros

    si  euros > 0  entonces

       dolares = (euros * TASA_CAMBIO)
       logaritmo = log10(dolares)

       escribir(euros + " euros equivalen a " + dolares + " dólares")
       escribir("El logaritmo en base 10 del resultado es " + logaritmo)

    sino
       escribir("Error: el monto en euros debe ser mayor que cero")
    FinSi

    escribir("El valor de euros es " + euros)
    escribir("El valor de TASA_CAMBIO es " + TASA_CAMBIO)
    escribir("El valor de dolares es " + dolares)
    escribir("El valor de logaritmo es " + logaritmo)

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
euros	100.0	100.0 euros equivalen a 108.0 dólares El logaritmo en base 10 del resultado es 2.0334 El valor de euros es 100.0 El valor de TASA_CAMBIO es 1.08 El valor de dolares es 108.0 El valor de logaritmo es 2.0334
dolares	108.0	
logaritmo	2.0334	

Prueba 2

Variable	Valor	Resultado
euros	0.0	Error: el monto en euros debe ser mayor que cero El valor de euros es 0.0 El valor de TASA_CAMBIO es 1.08 El valor de dolares es 0.0 El valor de logaritmo es 0.0
dolares	0.0	
logaritmo	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main()
{
    float TASA_CAMBIO = 1.08;
    float euros = 0.0;
    float dolares = 0.0;
    float logaritmo = 0.0;

    printf("Ingrese el monto en euros\n");
    scanf("%f",&euros);

    if (euros > 0)
    {
        dolares = euros * TASA_CAMBIO;
        logaritmo = log10(dolares);

        printf("%f euros equivalen a %f dólares\n", euros, dolares);
        printf("El logaritmo en base 10 del resultado es %f\n", logaritmo);
    }
    else
    {
        printf("Error: el monto en euros debe ser mayor que cero\n");
    }

    printf("El valor de euros es %f\n", euros);
    printf("El valor de TASA_CAMBIO es %f\n", TASA_CAMBIO);
    printf("El valor de dolares es %f\n", dolares);
    printf("El valor de logaritmo es %f\n", logaritmo);

    return 0;
}

```

Fig.34. Código en C del ejercicio 017

4.7. Salida del programa

```
Ingrese el monto en euros
100
100.000000 euros equivalen a 108.000008 dólares
El logaritmo en base 10 del resultado es 2.033424
El valor de euros es 100.000000
El valor de TASA_CAMBIO es 1.080000
El valor de dolares es 108.000008
El valor de logaritmo es 2.033424
```

Fig.35. Salida por terminal del programa

```
Ingrese el monto en euros
0
Error: el monto en euros debe ser mayor que cero
El valor de euros es 0.000000
El valor de TASA_CAMBIO es 1.080000
El valor de dolares es 0.000000
El valor de logaritmo es 0.000000
```

Fig.36. Salida por terminal del programa

Conclusiones:

-La constante de cambio TASA_CAMBIO = 1.08 encapsula el factor de conversión, haciendo el código más legible y mantenible

- La validación euros > 0 antes de calcular el logaritmo evita el error de calcular el log10 a un valor no positivo

EJ-018

En una empresa de transporte, diseñe un algoritmo que lea el peso (real, en kg) y la altura (real, en metros). Calcule el IMC con la expresion aritmetica (peso / (altura * altura)) y asignelo a imc. Use constantes simbolicas LIMITE_BAJO = 18.5 y LIMITE_ALTO = 25.0. Evalúe la expresion logica (imc >= LIMITE_BAJO AND imc <= LIMITE_ALTO). Muestre por pantalla el IMC calculado y si esta en rango normal. Ademas del calculo principal, calcule el resto de una division adicional. Incluya un mensaje de confirmacion al finalizar la operacion.

Análisis del problema

Datos Entrada:

Peso de la persona en kg (peso)

Altura de la persona en metros (altura)

Proceso:

1. Declarar las constantes simbólicas LIMITE_BAJO = 18.5 y LIMITE_ALTO = 25.0
2. Declarar las variables peso, altura, imc y resto e inicializarlas en 0.0
3. Recibir el peso en kg por teclado
4. Recibir la altura en metros por teclado
5. Verificar que el peso y la altura sean mayores que 0
6. Si no se cumple mostrar por terminal un mensaje de error y terminar el programa
7. Calcular el IMC con la fórmula imc = peso / (altura * altura)

8. Calcular el resto de dividir el peso entre 2 con la fórmula resto = peso - trunc(peso / 2) * 2
9. Mostrar por terminal el IMC calculado
10. Mostrar por terminal el resto de la división adicional
11. Verificar si se cumple la expresión lógica (imc >= LIMITE_BAJO AND imc <= LIMITE_ALTO)
12. Si se cumple, mostrar por terminal el mensaje “El IMC está en rango normal”
13. Si no se cumple, mostrar por terminal el mensaje “El IMC está fuera del rango normal”
14. Mostrar por terminal el mensaje de “operación finalizada correctamente”

Datos Salida:

Mensaje con el IMC calculado

Mensaje indicando si el IMC está en rango normal

Mensaje con el resto de la división adicional

Mensaje de confirmación al finalizar la operación

Mensaje de error si el peso o la altura no son mayores que 0

Diagrama de flujo

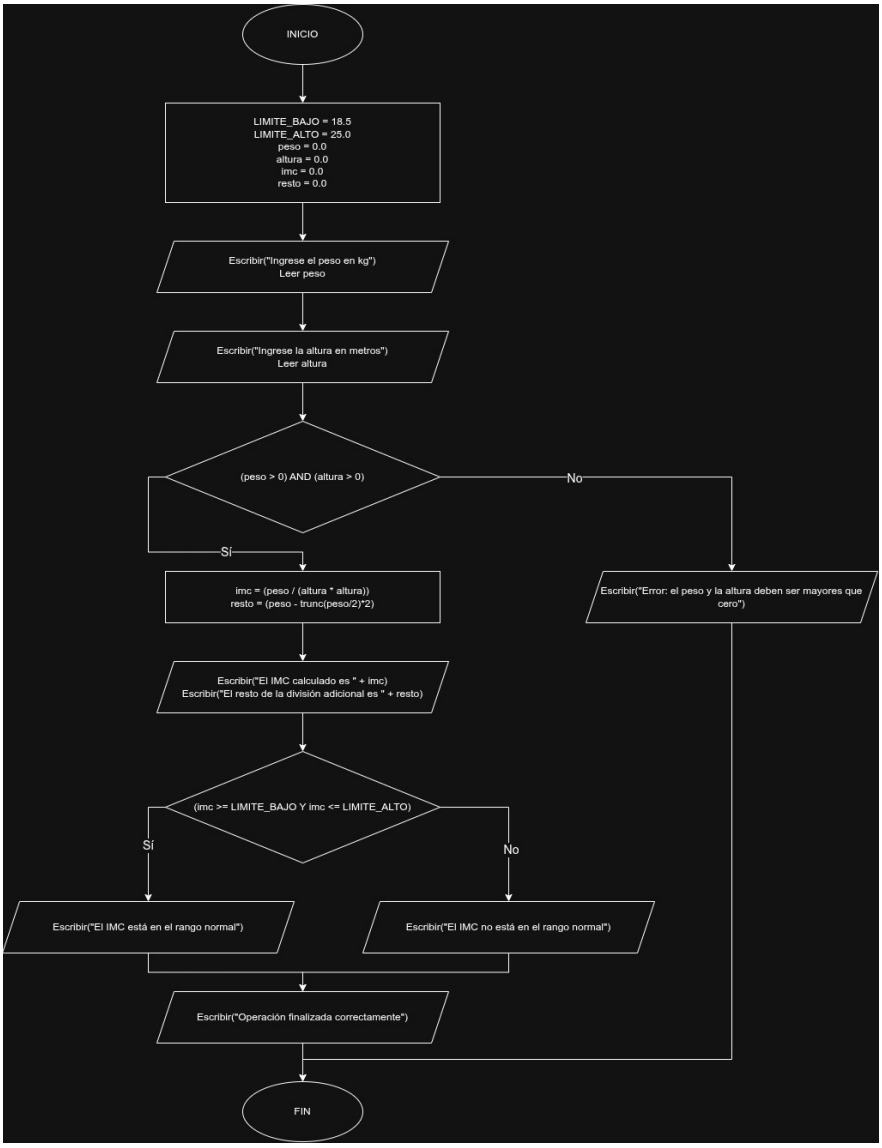


Fig.37. Diagrama de flujo del ejercicio 018

Pseudocódigo

```
Inicio

    LIMITE_BAJO = 18.5
    LIMITE_ALTO = 25.0

    peso = 0.0
    altura = 0.0
    imc = 0.0
    resto = 0.0

    escribir("Ingrese el peso en kg")
    leer peso

    escribir("Ingrese la altura en metros")
    leer altura

    si (peso > 0) AND (altura > 0) entonces

       imc = (peso / (altura * altura))

       resto = (peso - trunc(peso/2)*2)

       escribir("El IMC calculado es " + imc)
       escribir("El resto de la división adicional es "+resto)

       si  (imc >= LIMITE_BAJO Y imc <= LIMITE_ALTO)  entonces
          escribir("El IMC está en el rango normal")
       sino
```

```
        escribir("El IMC no está en el rango normal")
    FinSi

    escribir("Operación finalizada correctamente")

sino
    escribir("Error: el peso y la altura deben ser mayores que cero")
FinSi

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
peso	65.5	El IMC calculado es 22.6644 El resto de la división adicional es 1.5 El IMC está en el rango normal Operación finalizada correctamente
altura	1.70	
imc	22.6644	
resto	1.5	

Prueba 2

Variable	Valor	Resultado
peso	60.0	El IMC calculado es 26.6667 El resto de la división adicional es 0.0 El IMC no está en el rango normal Operación finalizada correctamente
altura	1.50	
imc	26.6667	
resto	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main()
{
    float LIMITE_BAJO = 18.5;
    float LIMITE_ALTO = 25.0;
    float peso = 0.0;
    float altura = 0.0;
    float imc = 0.0;
    float resto = 0.0;

    printf("Ingrese el peso en kg\n");
    scanf("%f",&peso);
    printf("Ingrese la altura en metros\n");
    scanf("%f",&altura);

    if (peso > 0 && altura > 0)
    {
        imc = peso / (altura * altura);
        resto = peso - trunc(peso / 2) * 2;

        printf("El IMC calculado es %f\n", imc);
        printf("El resto de la división adicional es %f\n", resto);

        if (imc >= LIMITE_BAJO && imc <= LIMITE_ALTO)
        {
            printf("El IMC está en el rango normal\n");
        }
        else
        {
            printf("El IMC no está en el rango normal\n");
        }

        printf("Operación finalizada correctamente\n");
    }
    else
    {
        printf("Error: el peso y la altura deben ser mayores que cero\n");
    }

    return 0;
}

```

Fig.38. Código en C del ejercicio 018

4.7. Salida del programa

```
Ingrese el peso en kg
65.6
Ingrese la altura en metros
1.70
El IMC calculado es 22.698961
El resto de la división adicional es 1.599998
El IMC está en el rango normal
Operación finalizada correctamente
```

Fig.39. Salida por terminal del programa

```
Ingrese el peso en kg
60
Ingrese la altura en metros
1.50
El IMC calculado es 26.666666
El resto de la división adicional es 0.000000
El IMC no está en el rango normal
Operación finalizada correctamente
```

Fig.40. Salida por terminal del programa

Conclusión:

-Aprendí a como calcular el indice de masa corporal para verificar el estado de salud

-En este ejercicio se utiliza el && como un clasificador de categorías, permitiendo saber si un elemento esta dentro del rango o no

EJ-021

En una panaderia, diseñe un algoritmo que lea un numero entero a por teclado. Evalúe las expresiones logicas (a > 0), (a < 0) y (a == 0). Asigne el resultado a tres variables booleanas: es_positivo, es_negativo, es_cero. Muestre por pantalla un mensaje indicando si el numero es positivo, negativo o cero. Justifique el tipo de dato de cada variable. Ademas, calcule el coseno del angulo resultado (si aplica). Agregue un mensaje indicando el tipo de operacion realizada.

Análisis del problema

Datos Entrada:

Número entero (a)

Variables usadas:

Nombre	Clase	Tipo de dato
a	Variable	Entero
es_positivo	Variable	Booleano
es_negativo	Variable	Booleano
es_cero	Variable	Booleano
valor	Variable	Entero
coseno	Variable	Real

Proceso:

1. Declarar la variable entera a, las variables booleanas es_positivo, es_negativo y es_cero, la variable entera valor y la variable real coseno
2. Recibir un número entero a
3. Evaluar la expresión lógica (a > 0) y asignar el resultado a es_positivo
4. Evaluar la expresión lógica (a < 0) y asignar el resultado a es_negativo
5. Evaluar la expresión lógica (a == 0) y asignar el resultado a es_cero
6. Verificar si es_positivo es verdadera
7. Si es verdadera, mostrar por terminal el mensaje “El número es positivo” y asignar a valor 1
8. Si es falsa, verificar si es_negativo es verdadera
9. Si es verdadera, mostrar por terminal el mensaje “El número es negativo” y asignar a valor 0
10. Si es falsa, mostrar por terminal el mensaje “El número es cero”
11. Calcular el coseno del ángulo resultado con la fórmula coseno = cos(valor).
12. Mostrar por terminal el coseno del ángulo resultado.
13. Mostrar por terminal un mensaje indicando que la operación realizada es el coseno

Datos Salida:

Mensaje indicando si el número es positivo, negativo o cero

Mensaje con el coseno del ángulo resultado

Mensaje indicando que la operación realizada es el coseno

Diagrama de flujo

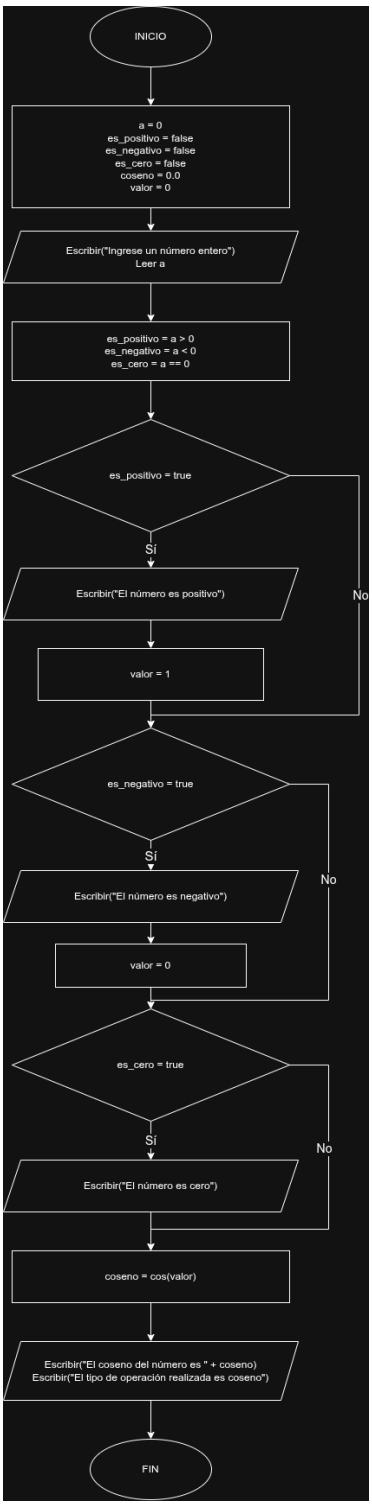


Fig.41. Diagrama de flujo del ejercicio 021

Pseudocódigo

Inicio

```
a = 0
es_positivo = false
es_negativo = false
es_cero = false
coseno = 0.0
valor = 0

escribir("Ingrese un número entero")
leer a

es_positivo = a > 0
es_negativo = a < 0
es_cero = a == 0

si es_positivo = true entonces
escribir("El número es positivo")
valor = 1
FinSi

si es_negativo = true entonces
escribir("El número es negativo")
valor = 0
FinSi

si es_cero = true entonces
escribir("El número es cero")
FinSi

coseno = cos(valor)
```

escribir("El coseno del número es " + coseno)

escribir("El tipo de operación realizada es coseno")

Fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
a	5	El número es positivo El coseno del número es 0.5403 El tipo de operación realizada es coseno
es_positivo	true	
es_negativo	false	
es_cero	false	
valor	1	
coseno	0.5403	

Prueba 2

Variable	Valor	Resultado
a	-3	El número es negativo El coseno del número es 1.0 El tipo de operación realizada es coseno
es_positivo	false	
es_negativo	true	
es_cero	false	
valor	0	
coseno	1.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

int main()
{
    int a = 0;
    bool es_positivo = false;
    bool es_negativo = false;
    bool es_cero = false;
    float coseno = 0.0;
    int valor = 0;

    printf("Ingrese un número entero\n");
    scanf("%d",&a);

    es_positivo = a > 0;
    es_negativo = a < 0;
    es_cero = a == 0;

    if (es_positivo == true)
    {
        printf("El número es positivo\n");
        valor = 1;
    }

    if (es_negativo == true)
    {
        printf("El número es negativo\n");
        valor = 0;
    }

    if (es_cero == true)
    {
        printf("El número es cero\n");
    }

    coseno = cos(valor);

    printf("El coseno del número es %f\n", coseno);
    printf("El tipo de operación realizada es coseno\n");

    return 0;
}

```

Fig.42. Código en C del ejercicio 021

4.7. Salida del programa

```
Ingrese un número entero
5
El número es positivo
El coseno del número es 0.540302
El tipo de operación realizada es coseno
```

Fig.42. Salida por terminal del programa

```
Ingrese un número entero
-3
El número es negativo
El coseno del número es 1.000000
El tipo de operación realizada es coseno
```

Fig.43. Salida por terminal del programa

Conclusiones:

-Este ejercicio muestra como una condición se puede evaluar para 3 casos diferentes como es el caso de es_positivo, es_negativo y es_cero

-El coseno del valor (1, 0 o 1 para negativo) muestra cómo una función matemática puede aplicarse al resultado de una clasificación

EJ-023

En una farmacia, construya un algoritmo que lea un numero entero (1 a 12) por teclado. Use constantes literales 1 y 12 para validar el rango. Evalúe la expresión lógica (mes >= 1 AND mes <= 12). Use expresiones relacionales para asignar el nombre del mes a una variable cadena. Muestre por pantalla el nombre del mes o un mensaje de error. Ademas del calculo principal, determine el doble del resultado. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

Datos Entrada:

Número entero del mes (mes)

Proceso:

1. Declarar la variable entera mes, la variable de tipo cadena nombre_mes y la variable entera doble
2. Recibir un número entero por teclado y almacenarlo en mes
3. Verificar que se cumpla la expresión lógica (mes >= 1 AND mes <= 12)
4. Si no se cumple mostrar por terminal el mensaje “El número no corresponde a un mes válido” y terminar el programa
5. Evaluar la expresión relacional (mes == 1) y si se cumple asignar “Enero” a nombre_mes
6. Evaluar la expresión relacional (mes == 2) y si se cumple asignar “Febrero” a nombre_mes
7. Evaluar la expresión relacional (mes == 3) y si se cumple asignar “Marzo” a nombre_mes
8. Evaluar la expresión relacional (mes == 4) y si se cumple asignar “Abril” a nombre_mes
9. Evaluar la expresión relacional (mes == 5) y si se cumple asignar “Mayo” a nombre_mes
10. Evaluar la expresión relacional (mes == 6) y si se cumple asignar “Junio” a nombre_mes
11. Evaluar la expresión relacional (mes == 7) y si se cumple asignar “Julio” a nombre_mes

12. Evaluar la expresión relacional (mes == 8) y si se cumple asignar “Agosto” a nombre_mes
13. Evaluar la expresión relacional (mes == 9) y si se cumple asignar “Septiembre” a nombre_mes
14. Evaluar la expresión relacional (mes == 10) y si se cumple asignar “Octubre” a nombre_mes
15. Evaluar la expresión relacional (mes == 11) y si se cumple asignar “Noviembre” a nombre_mes
16. Evaluar la expresión relacional (mes == 12) y si se cumple asignar “Diciembre” a nombre_mes
17. Determinar el doble del número ingresado con la fórmula doble = mes * 2
18. Mostrar por terminal el nombre del mes
19. Mostrar por terminal el doble del número ingresado
20. Mostrar por terminal un mensaje de confirmación de que la operación finalizó correctamente

Datos Salida:

Mensaje con el nombre del mes

Mensaje con el doble del número ingresado

Mensaje de confirmación al finalizar la operación

Mensaje de error si el número no está entre 1 y 12

Diagrama de flujo

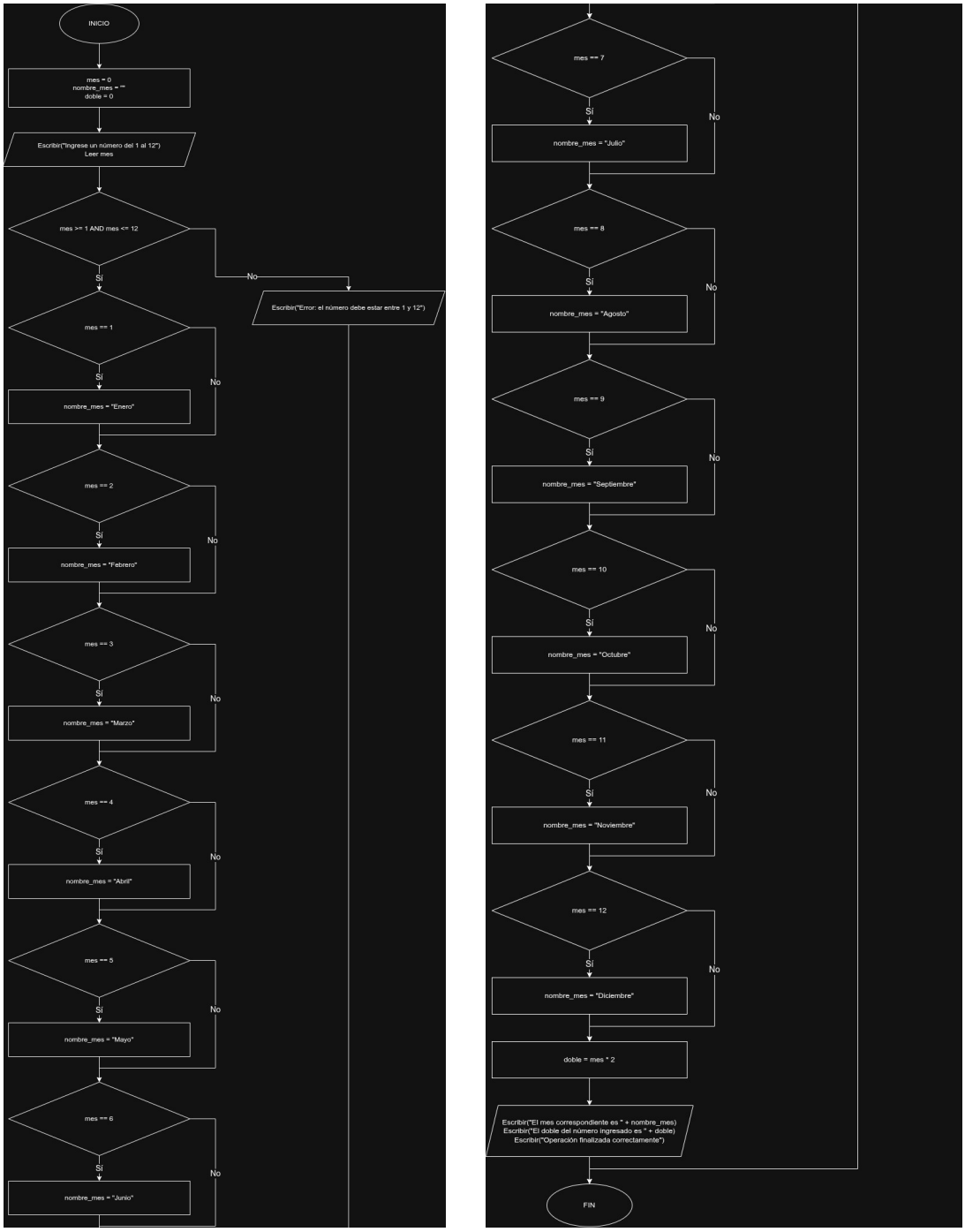


Fig.44. Diagrama de flujo del ejercicio 023

Pseudocódigo

Inicio

mes = 0

```
nombre_mes = ""

doble = 0


escribir("Ingrese un número del 1 al 12")

leer mes


si mes >= 1 AND mes <= 12 entonces


si mes == 1 entonces
        nombre_mes = "Enero"
FinSi


si mes == 2 entonces
        nombre_mes = "Febrero"
FinSi


si mes == 3 entonces
        nombre_mes = "Marzo"
FinSi

si mes == 4 entonces
        nombre_mes = "Abril"
FinSi


si mes == 5 entonces
        nombre_mes = "Mayo"
FinSi


si mes == 6 entonces
        nombre_mes = "Junio"
FinSi


si mes == 7 entonces
        nombre_mes = "Julio"
FinSi


si mes == 8 entonces
        nombre_mes = "Agosto"
FinSi
```

```
si mes == 9 entonces
        nombre_mes = "Septiembre"
FinSi


si mes == 10 entonces
        nombre_mes = "Octubre"
FinSi



si mes == 11 entonces
        nombre_mes = "Noviembre"
FinSi


si mes ==12 entonces
        nombre_mes = "Diciembre"
FinSi

doble = mes * 2

escribir("El mes correspondiente es " + nombre_mes)

escribir("El doble del número ingresado es " + doble)

escribir("Operación finalizada correctamente")

sino

escribir("Error: el número debe estar entre 1 y 12")

FinSi
```

Fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
mes	3	El mes correspondiente es Marzo El doble del número ingresado es 6 Operación finalizada correctamente
nombre_mes	"Marzo"	
doble	6	

Prueba 2

Variable	Valor	Resultado
mes	15	Error: el número debe estar entre 1 y 12
nombre_mes	""	
doble	0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <string.h>

int main()
{
    int mes = 0;
    char nombre_mes[20] = "";
    int doble = 0;

    printf("Ingrese un número del 1 al 12\n");
    scanf("%d",&mes);

    if (mes >= 1 && mes <= 12)
    {
        if (mes == 1)
        {
            strcpy(nombre_mes, "Enero");
        }
        if (mes == 2)
        {
            strcpy(nombre_mes, "Febrero");
        }
        if (mes == 3)
        {
            strcpy(nombre_mes, "Marzo");
        }
        if (mes == 4)
        {
            strcpy(nombre_mes, "Abril");
        }
        if (mes == 5)
        {
            strcpy(nombre_mes, "Mayo");
        }
        if (mes == 6)
        {
            strcpy(nombre_mes, "Junio");
        }
        if (mes == 7)
        {
            strcpy(nombre_mes, "Julio");
        }
        if (mes == 8)
        {
            strcpy(nombre_mes, "Agosto");
        }
        if (mes == 9)
        {
            strcpy(nombre_mes, "Septiembre");
        }
        if (mes == 10)
        {
            strcpy(nombre_mes, "Octubre");
        }
        if (mes == 11)
        {
            strcpy(nombre_mes, "Noviembre");
        }
        if (mes == 12)
        {
            strcpy(nombre_mes, "Diciembre");
        }

        doble = mes * 2;

        printf("El mes correspondiente es %s\n", nombre_mes);
        printf("El doble del número ingresado es %d\n", doble);
        printf("Operación finalizada correctamente\n");
    }
    else
    {
        printf("Error: el número debe estar entre 1 y 12\n");
    }

    return 0;
}
```

Fig.45. Código en C del ejercicio 023

4.7. Salida del programa

```
Ingrese un número del 1 al 12
3
El mes correspondiente es Marzo
El doble del número ingresado es 6
Operación finalizada correctamente
```

Fig.46. Salida por terminal del programa

```
Ingrese un número del 1 al 12
15
Error: el número debe estar entre 1 y 12
```

Fig.47. Salida por terminal del programa

Conclusiones:

- La validación de rango mes >= 1 AND mes <= 12 con literales asegura que solo se procesen números correspondientes a meses reales

-La asignación del nombre del mes mediante comparaciones secuenciales es una alternativa simple al uso de arreglos o switch

EJ-024

En una escuela, diseñe un algoritmo que lea un numero entero y por teclado. Use constantes simbolicas MIN = 10 y MAX = 100. Evalúe la expresion logica (y >= MIN AND y <= MAX). Asigne el resultado a una variable booleana en_rango. Muestre por pantalla si el numero esta dentro o fuera del rango. Identifique cada tipo de dato. Ademas del calculo principal, multiplique el resultado por 1.5 como bono. Agregue validaciones para que no haya resultados negativos.

Análisis del problema

Datos Entrada:

Número entero (y)

Variables usadas:

Nombre	Clase	Tipo de dato
y	Variable	Entero
en_rango	Variable	Booleano
bono	Variable	Real
MIN	Constante simbólica	Entero
MAX	Constante simbólica	Entero

Proceso:

1. Declarar las constantes simbólicas MIN = 10 y MAX = 100
2. Declarar la variable entera y, la variable booleana en_rango y la variable real bono
3. Recibir un número entero por teclado y almacenarlo en y
4. Verificar que el número ingresado no sea negativo usando la expresión relacional (y>=0)
5. Si no se cumple mostrar por terminal un mensaje de error y terminar el programa
6. Evaluar la expresión lógica (y >= MIN AND y <= MAX) y asignar el resultado a en_rango

7. Determinar el bono con la fórmula bono = y * 1.5
8. Verificar si en_rango es verdadera
9. Si es verdadera, mostrar por terminal el mensaje “El número está dentro del rango”
10. Si es falsa, mostrar por terminal el mensaje “El número está fuera del rango”
11. Mostrar por terminal el bono
12. Mostrar por terminal un mensaje de confirmación de que la operación finalizó correctamente

Datos Salida:

Mensaje indicando si el número está dentro o fuera del rango

Mensaje con el bono

Mensaje de confirmación al finalizar la operación

Mensaje de error si el número es negativo

Diagrama de flujo

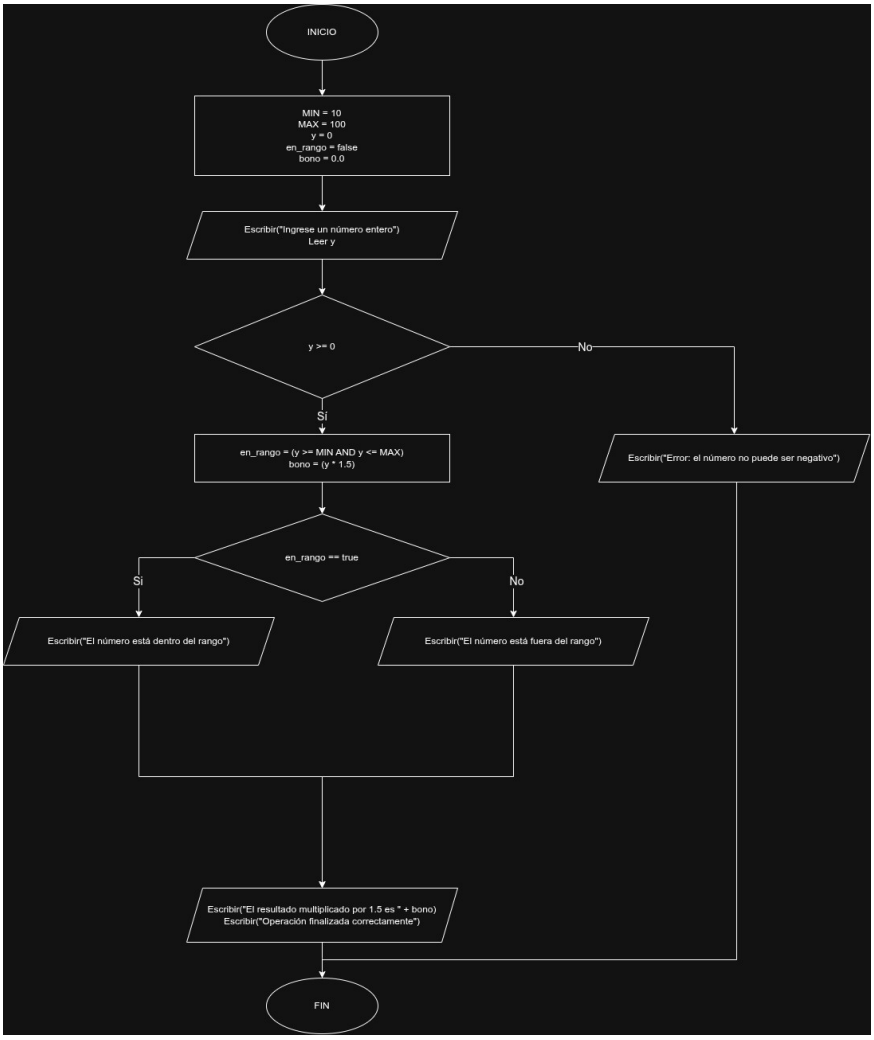


Fig.48. Diagrama de flujo del ejercicio 024

Pseudocódigo

```
Inicio

    MIN = 10
    MAX = 100
    y = 0
    en_rango = false
    bono = 0.0

    escribir("Ingrese un número entero")
    leer y

    si y >= 0 entonces

       en_rango = (y >= MIN AND y <= MAX)
       bono = (y * 1.5)

       si en_rango == true entonces
          escribir("El número está dentro del rango")
       sino
          escribir("El número está fuera del rango")
       FinSi

       escribir("El resultado multiplicado por 1.5 es " + bono)
       escribir("Operación finalizada correctamente")

    sino
       escribir("Error: el número no puede ser negativo")
    FinSi
```

Fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
y	50	El número está dentro del rango El resultado multiplicado por 1.5 es 75.0 Operación finalizada correctamente
en_rango	true	
bono	75.0	

Prueba 2

Variable	Valor	Resultado
y	5	El número está fuera del rango El resultado multiplicado por 1.5 es 7.5 Operación finalizada correctamente
en_rango	false	
bono	7.5	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    int MIN = 10;
    int MAX = 100;
    int y = 0;
    bool en_rango = false;
    float bono = 0.0;

    printf("Ingrese un número entero\n");
    scanf("%d",&y);

    if (y >= 0)
    {
        en_rango = (y >= MIN && y <= MAX);
        bono = y * 1.5;

        if (en_rango == true)
        {
            printf("El número está dentro del rango\n");
        }
        else
        {
            printf("El número está fuera del rango\n");
        }

        printf("El resultado multiplicado por 1.5 es %f\n", bono);
        printf("Operación finalizada correctamente\n");
    }
    else
    {
        printf("Error: el número no puede ser negativo\n");
    }

    return 0;
}
```

Fig.49. Código en C del ejercicio 024

4.7. Salida del programa

```
Ingrese un número entero
50
El número está dentro del rango
El resultado multiplicado por 1.5 es 75.000000
Operación finalizada correctamente
```

Fig.50. Salida por terminal del progama

```
Ingrese un número entero
5
El número está fuera del rango
El resultado multiplicado por 1.5 es 7.500000
Operación finalizada correctamente
```

Fig. 51. Salida por terminal del programa

Conclusiones:

-La combinación de constantes MIN y MAX con un operador AND define un intervalo de validación reutilizable

-Multiplicar el número por 1.5 como bono muestra cómo un factor puede adjustar un valor según una regla de negocio

EJ-028

En un restaurante, cree un algoritmo que lea cinco numeros enteros por teclado. Use la constante literal 5 como divisor. Calcule la suma y el promedio con expresiones aritmeticas. Asigne los resultados a suma y promedio. Evalúe la expresión relacional (suma > 0). Muestre por pantalla la suma y el promedio. Identifique el tipo de dato de cada variable. Ademas, calcule el valor absoluto de la diferencia entre dos resultados. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

Datos Entrada:

Primer número entero (n1)

Segundo número entero (n2)

Tercer número entero (n3)

Cuarto número entero (n4)

Quinto número entero (n5)

Variables usadas:

Nombre	Clase	Tipo de dato
n1	Variable	Entero
n2	Variable	Entero
n3	Variable	Entero
n4	Variable	Entero
n5	Variable	Entero
suma	Variable	Entero
promedio	Variable	Real
diferencia	Variable	Real
5	Constante literal	Entero

Proceso:

1. Declarar las variables enteras n1, n2, n3, n4, n5 y suma, y las variables reales promedio y diferencia
2. Recibir el primer número n1
3. Recibir el segundo número n2
4. Recibir el tercer número n3

5. Recibir el cuarto número n4
6. Recibir el quinto número n5.
7. Calcular la suma con la fórmula suma = n1 + n2 + n3 + n4 + n5.
8. Calcular el promedio con la fórmula promedio = suma / 5.0.
9. Calcular el valor absoluto de la diferencia entre la suma y el promedio con la fórmula diferencia = abs(suma - promedio)
10. Mostrar por terminal la suma
11. Mostrar por terminal el promedio
12. Verificar si se cumple la expresión relacional (suma > 0)
13. Si se cumple, mostrar por terminal el mensaje “La suma es positiva”
14. Si no se cumple, verificar si la suma es igual a 0
15. Si es igual a 0, mostrar por terminal el mensaje “La suma es cero”
16. Si es distinta de 0, mostrar por terminal el mensaje “La suma es negativa”
17. Mostrar por terminal el valor absoluto de la diferencia entre la suma y el promedio
18. Mostrar por terminal un mensaje de confirmación de que la operación finalizó correctamente

Datos Salida:

Mensaje con la suma

Mensaje con el promedio

Mensaje indicando si la suma es positiva, cero o negativa

Mensaje con el valor absoluto de la diferencia entre la suma y el promedio

Mensaje de confirmación al finalizar la operación

Diagrama de flujo

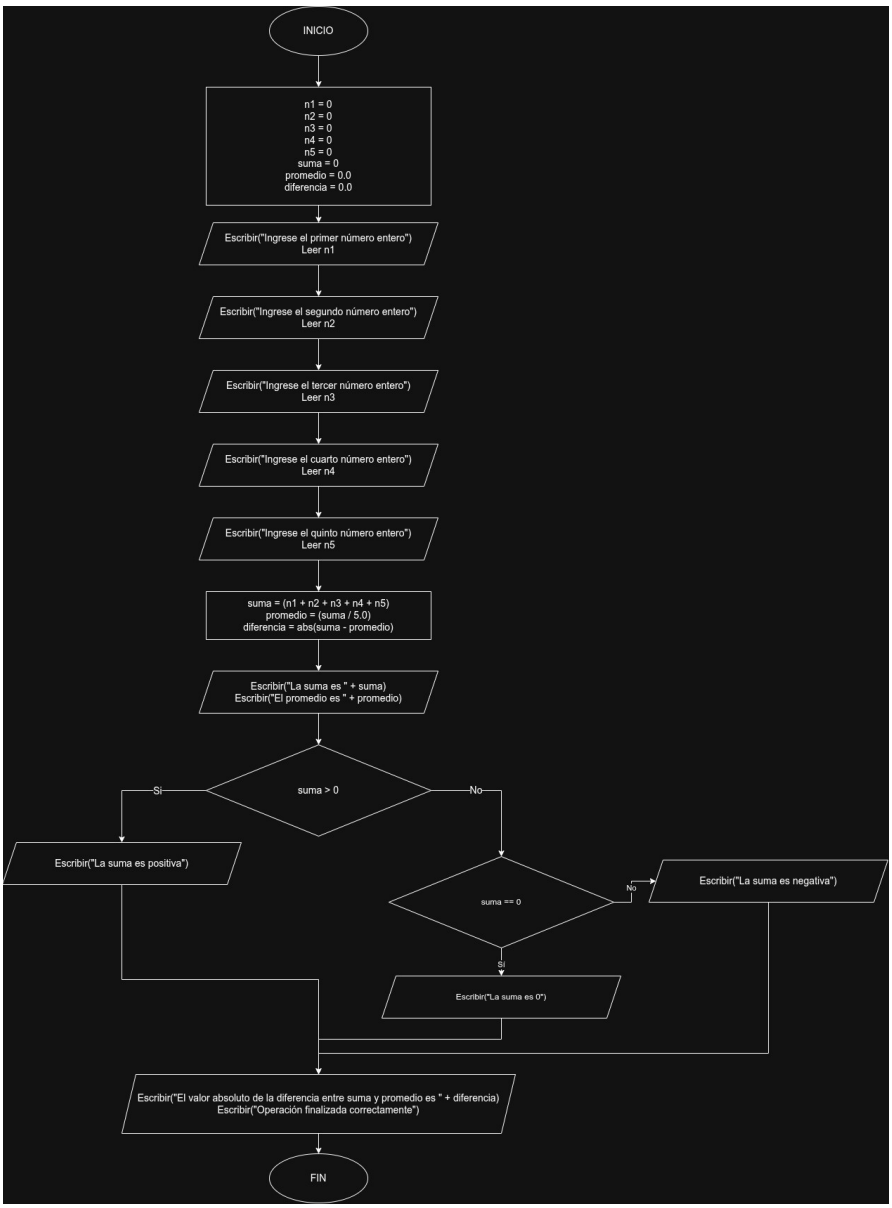


Fig.52. Diagrama de flujo del ejercicio 028

Pseudocódigo

Inicio

n1 = 0

n2 = 0

n3 = 0

n4 = 0

```
n5 = 0
suma = 0
promedio = 0.0
diferencia = 0.0

escribir("Ingrese el primer número entero")
leer n1
escribir("Ingrese el segundo número entero")
leer n2
escribir("Ingrese el tercer número entero")
leer n3
escribir("Ingrese el cuarto número entero")
leer n4
escribir("Ingrese el quinto número entero")
leer n5

suma = (n1 + n2 + n3 + n4 + n5)
promedio = (suma / 5.0)
diferencia = abs(suma - promedio)

escribir("La suma es " + suma)
escribir("El promedio es " + promedio)
si suma > 0 entonces
        escribir("La suma es positiva")
sino si suma == 0 entonces
        escribir("La suma es 0")
sino
        escribir("La suma es negativa")
FinSi
escribir("El valor absoluto de la diferencia entre suma y promedio es " + diferencia)
```

```
    escribir("Operación finalizada correctamente")

Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
n1	4	La suma es 30 El promedio es 6.0 La suma es positiva El valor absoluto de la diferencia entre suma y promedio es 24.0 Operación finalizada correctamente
n2	7	
n3	10	
n4	3	
n5	6	
suma	30	
promedio	6.0	
diferencia	24.0	

Prueba 2

Variable	Valor	Resultado
n1	-2	La suma es -20 El promedio es -4.0 La suma es negativa El valor absoluto de la diferencia entre suma y promedio es 16.0 Operación finalizada correctamente
n2	-3	
n3	-4	
n4	-5	
n5	-6	
suma	-20	
promedio	-4.0	
diferencia	16.0	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main()
{
    int n1 = 0;
    int n2 = 0;
    int n3 = 0;
    int n4 = 0;
    int n5 = 0;
    int suma = 0;
    float promedio = 0.0;
    float diferencia = 0.0;

    printf("Ingrese el primer número entero\n");
    scanf("%d",&n1);
    printf("Ingrese el segundo número entero\n");
    scanf("%d",&n2);
    printf("Ingrese el tercer número entero\n");
    scanf("%d",&n3);
    printf("Ingrese el cuarto número entero\n");
    scanf("%d",&n4);
    printf("Ingrese el quinto número entero\n");
    scanf("%d",&n5);

    suma = n1 + n2 + n3 + n4 + n5;
    promedio = suma / 5.0;
    diferencia = fabs(suma - promedio);

    printf("La suma es %d\n", suma);
    printf("El promedio es %f\n", promedio);

    if (suma > 0)
    {
        printf("La suma es positiva\n");
    }
    else if (suma == 0)
    {
        printf("La suma es 0\n");
    }
    else
    {
        printf("La suma es negativa\n");
    }

    printf("El valor absoluto de la diferencia entre suma y promedio es %f\n", diferencia);
    printf("Operación finalizada correctamente\n");

    return 0;
}
```

Fig.53. Código en C del ejercicio 028

4.7. Salida del programa

```
Ingrese el primer número entero
4
Ingrese el segundo número entero
7
Ingrese el tercer número entero
10
Ingrese el cuarto número entero
3
Ingrese el quinto número entero
6
La suma es 30
El promedio es 6.000000
La suma es positiva
El valor absoluto de la diferencia entre suma y promedio es 24.000000
Operación finalizada correctamente
```

Fig.54. Salida por terminal del prorgama

```
Ingrese el primer número entero
-2
Ingrese el segundo número entero
-3
Ingrese el tercer número entero
-4
Ingrese el cuarto número entero
-5
Ingrese el quinto número entero
-6
La suma es -20
El promedio es -4.000000
La suma es negativa
El valor absoluto de la diferencia entre suma y promedio es 16.000000
Operación finalizada correctamente
```

Fig.55. Salida por terminal del programa

Conclusione:

-La lectura de múltiples valores seguidos y su acumulación en una variable de suma es la base de cualquier algoritmo de aggregación

- El valor absoluto de la diferencia entre suma y promedio (abs(suma - promedio)) muestra cómo medir la dispersión entre dos estadísticas

EJ-076

En una tienda de ropa, construya un algoritmo que calcule el impuesto sobre la renta progresivo. Lea el ingreso anual (real). Use constantes simbolicas TRAMO_1 = 10000, TRAMO_2 = 30000, TRAMO_3 = 60000, TASA_1 = 0.10, TASA_2 = 0.15, TASA_3 = 0.20, TASA_4 = 0.30. Evalúe las expresiones logicas relacionales para determinar los tramos aplicables. Calcule el impuesto con expresiones aritmeticas por tramo. Asignelo a impuesto_total. Muestre por pantalla el desglose. Ademas, calcule el valor minimo de todas las variables utilizadas. Identifique explicitamente los Datos Entrada y la informacion de salida.

Análisis del problema

Datos Entrada:

Ingreso anual (ingreso)

Proceso:

1. Declarar las constantes simbólicas TRAMO_1 = 10000, TRAMO_2 = 30000, TRAMO_3 = 60000, TASA_1 = 0.10. TASA_2 = 0.15, TASA_3 = 0.20 y TASA_4 = 0.30
2. Declarar las variables ingreso, impuesto_total, impuesto1, impuesto2, impuesto3, impuesto4, minimo como real
3. Recibir ingreso por teclado
4. Verificar ingreso < 0
5. Si se cumple mostrar mensaje de error y finalizar el programa
6. Evaluar las expresiones lógicas relacionales para determinar los tramos aplicables
7. Calcular el impuesto individual de cada tramo aplicable
8. Calcular el impuesto total con la suma de los impuestos individuales
9. Mostrar un mensaje con el impuesto obtenido por cada tramo y el impuesto total
10. Asignar a minimo el valor de ingreso
11. Calcular el valor minimo de las variables utilizadas realizando de forma secuencial comparaciones entre el valor minimo actual y las demás variables utilizadas, asignando el menor valor a la variable minimo en cada caso (impuesto1, impuesto2, impuesto3, impuesto4, impuesto_total)
12. Mostrar un mensaje por terminal con el valor mínimo de las variables utilizadas

Datos Salida:

Mensaje mostrando el impuesto del tramo 1

Mensaje mostrando el impuesto del tramo 2

Mensaje mostrando el impuesto del tramo 3

Mensaje mostrando el impuesto del tramo 4

Mensaje mostrando el impuesto total obtenido

Mensaje mostrando el valor minimo de las variables utilizadas

Diagrama de flujo

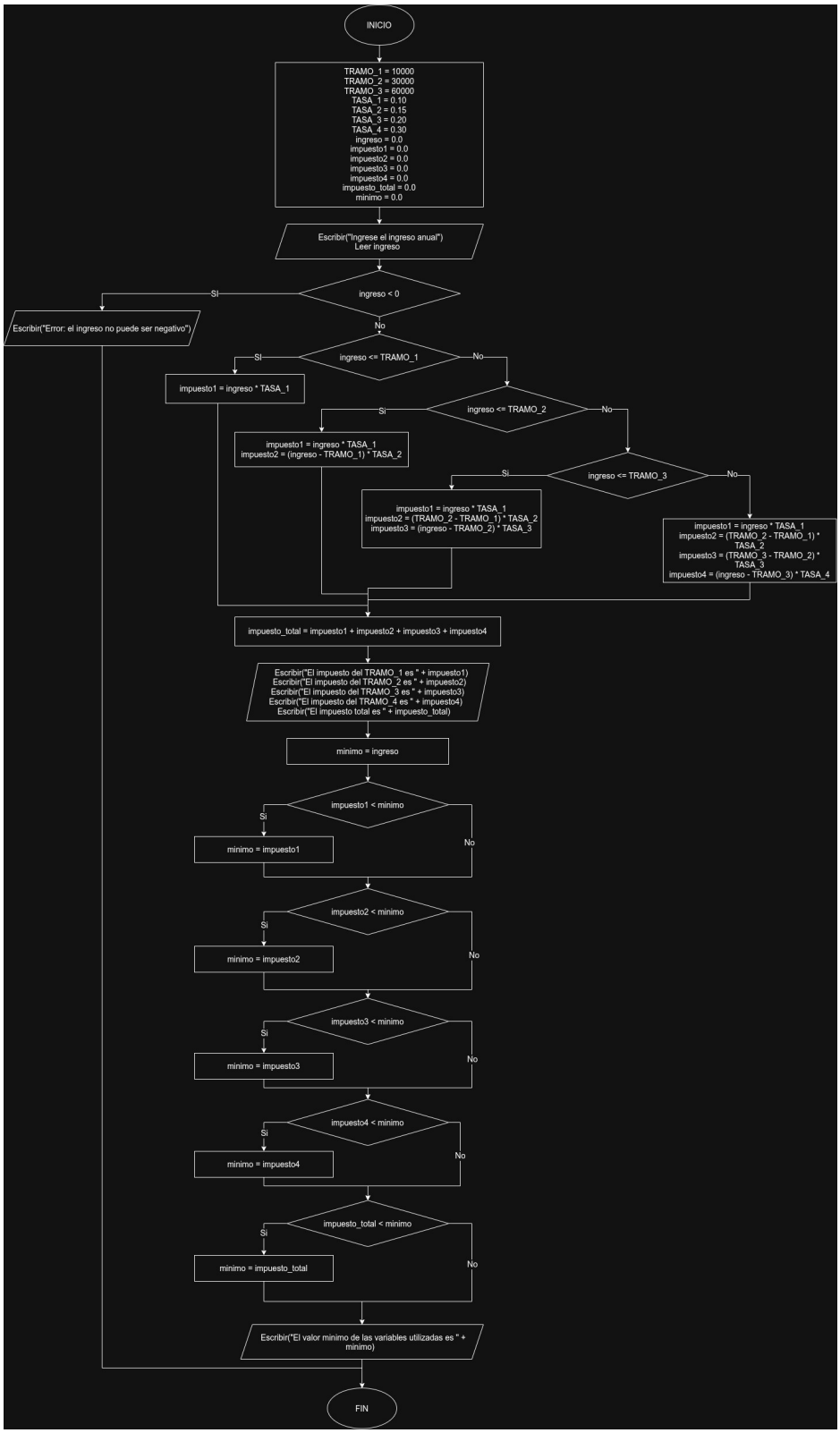


Fig.56. Diagrama de flujo del ejercicio 076

Pseudocódigo

```
Incio
        TRAMO_1 = 10000
        TRAMO_2 = 30000
        TRAMO_3 = 60000
        TASA_1 = 0.10
        TASA_2 = 0.15
        TASA_3 = 0.20
        TASA_4 = 0.30

        ingreso = 0.0
        impuesto1 = 0.0
        impuesto2 = 0.0
        impuesto3 = 0.0
        impuesto4 = 0.0
        impuesto_total = 0.0
        minimo = 0.0

        escribir("Ingrese el ingreso anual")
        leer ingreso
        si ingreso < 0 entonces
                escribir("Error: el ingreso no puede ser negativo")
        sino
                si ingreso <= TRAMO_1 entonces
                        impuesto1 = ingreso * TASA_1
                sino si ingreso <= TRAMO_2 entonces
                        impuesto1 = ingreso * TASA_1
                        impuesto2 = (ingreso – TRAMO_1) * TASA_2
                sino si ingreso <= TRAMO_3 entonces
                        impuesto1 = ingreso * TASA_1
```

```
            impuesto2 = (TRAMO_2 – TRAMO_1) * TASA_2
            impuesto3 = (ingreso – TRAMO_2) * TASA_3
sino
            impuesto1 = ingreso * TASA_1
            impuesto2 = (TRAMO_2 – TRAMO_1) * TASA_2
            impuesto3 = (TRAMO_3 – TRAMO_2) * TASA_3
            impuesto4 = (ingreso – TRAMO_3) * TASA_4
FinSi


impuesto_total = impuesto1 + impuesto2 + impuesto3 + impuesto4
minimo = ingreso
si impuesto1 < minimo entonces
            minimo = impuesto1
FinSi
si impuesto2 < minimo entonces
            minimo = impuesto2
FinSi
si impuesto3 < minimo entonces
            minimo = impuesto3
FinSi
si impuesto4 < minimo entonces
            minimo = impuesto4
FinSi
si impuesto_total < minimo entonces
            minimo = impuesto_total
FinSi

escribir("El impuesto del TRAMO_1 es " + impuesto1)
escribir("El impuesto del TRAMO_2 es " + impuesto2)
```

```
            escribir("El impuesto del TRAMO_3 es " + impuesto3)
            escribir("El impuesto del TRAMO_4 es " + impuesto4)
            escribir("El impuesto total es " + impuesto_total)
            escribir("El valor minimo de las variables utilizadas es "+minimo)
        FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
ingreso	25000.0	El impuesto del TRAMO_1 es 2500.0 El impuesto del TRAMO_2 es 2250.0 El impuesto del TRAMO_3 es 0.0 El impuesto del TRAMO_4 es 0.0 El impuesto total es 4750.0 El valor minimo de las variables utilizadas es 0.0
impuesto1	2500.0	
impuesto2	2250.0	
impuesto3	0.0	
impuesto4	0.0	
impuesto_total	4750.0	
minimo	0.0	

Prueba 2

Variable	Valor	Resultado
ingreso	8000.0	El impuesto del TRAMO_1 es 800.0 El impuesto del TRAMO_2 es 0.0 El impuesto del TRAMO_3 es 0.0 El impuesto del TRAMO_4 es 0.0 El impuesto total es 800.0 El valor minimo de las variables utilizadas es 0.0
impuesto1	800.0	
impuesto2	0.0	
impuesto3	0.0	
impuesto4	0.0	
impuesto_total	800.0	
minimo	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{
    int TRAMO_1 = 10000;
    int TRAMO_2 = 30000;
    int TRAMO_3 = 60000;
    float TASA_1 = 0.10;
    float TASA_2 = 0.15;
    float TASA_3 = 0.20;
    float TASA_4 = 0.30;

    float ingreso = 0.0;
    float impuesto1 = 0.0;
    float impuesto2 = 0.0;
    float impuesto3 = 0.0;
    float impuesto4 = 0.0;
    float impuesto_total = 0.0;
    float minimo = 0.0;

    printf("Ingrese el ingreso anual\n");
    scanf("%f",&ingreso);

    if (ingreso < 0)
    {
        printf("Error: el ingreso no puede ser negativo\n");
    }

    else
    {
        if (ingreso <= TRAMO_1)
        {
            impuesto1 = ingreso * TASA_1;
        }
        else if (ingreso <= TRAMO_2)
        {
            impuesto1 = ingreso * TASA_1;
            impuesto2 = (ingreso - TRAMO_1) * TASA_2;
        }
        else if (ingreso <= TRAMO_3)
        {
            impuesto1 = ingreso * TASA_1;
            impuesto2 = (TRAMO_2 - TRAMO_1) * TASA_2;
            impuesto3 = (ingreso - TRAMO_2) * TASA_3;
        }
        else
        {
            impuesto1 = ingreso * TRAMO_1;
            impuesto2 = (TRAMO_2 - TRAMO_1) * TASA_2;
            impuesto3 = (TRAMO_3 - TRAMO_2) * TASA_3;
            impuesto4 = (ingreso - TRAMO_3) * TASA_4;
        }

```

Fig.57. Código en C del ejercicio 076

```c
        impuesto_total = impuesto1 + impuesto2 + impuesto3 + impuesto4;

        minimo = ingreso;

        if (impuesto1 < minimo)
        {
            minimo = impuesto1;
        }
        if (impuesto2 < minimo)
        {
            minimo = impuesto2;
        }
        if (impuesto3 < minimo)
        {
            minimo = impuesto3;
        }
        if (impuesto4 < minimo)
        {
            minimo = impuesto4;
        }
        if (impuesto_total < minimo)
        {
            minimo = impuesto_total;
        }

        printf("El impuesto del tramo 1 es %.2f\n",impuesto1);
        printf("El impuesto del tramo 2 es %.2f\n",impuesto2);
        printf("El impuesto del tramo 3 es %.2f\n",impuesto3);
        printf("El impuesto del tramo 4 es %.2f\n\n",impuesto4);
        printf("El impuesto total es %.2f\n",impuesto_total);
        printf("El valor minimo de las variables utilizadas es %.2f\n",minimo);




    }


    return 0;
}
```

Fig.58. Código en C del ejercicio 076

4.7. Salida del programa

```
Ingrese el ingreso anual
25000
El impuesto del tramo 1 es 2500.00
El impuesto del tramo 2 es 2250.00
El impuesto del tramo 3 es 0.00
El impuesto del tramo 4 es 0.00

El impuesto total es 4750.00
El valor minimo de las variables utilizadas es 0.00
```

Fig.59. Salida por terminal del programa

```
Ingrese el ingreso anual
8000
El impuesto del tramo 1 es 800.00
El impuesto del tramo 2 es 0.00
El impuesto del tramo 3 es 0.00
El impuesto del tramo 4 es 0.00

El impuesto total es 800.00
El valor minimo de las variables utilizadas es 0.00
```

Fig.60. Salida por terminal del programa

Conclusiones:

-El algoritmo de tramos progresivos muestra cómo los impuestos se calculan por rangos aplicando tasas diferentes a cada porción del ingreso

-El cálculo del valor mínimo de todas las variables incluidas las tasas y los impuestos parciales, demostrando una técnica de comparación que puede usarse en cualquier contexto

EJ-077

En una farmacia, cree un algoritmo que simule un sistema de puntuacion. Lea los puntos obtenidos (entero), el tiempo jugado (real, en minutos) y el nivel alcanzado (entero). Use constantes simbolicas BONO_TIEMPO = 0.10, BONO_NIVEL = 100, TIEMPO_MIN = 30.0, NIVEL_MAX = 10. Calcule: bono_tiempo = puntos * BONO_TIEMPO * (tiempo < TIEMPO_MIN ? 1 : 0); bono_nivel = nivel * BONO_NIVEL; puntuacion_total = puntos + bono_tiempo + bono_nivel. Evalúe la expresion logica (nivel >= 1 AND nivel <= NIVEL_MAX). Muestre por pantalla la puntuacion total. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de inicializar todas las variables antes de usarlas.

Análisis del problema

Datos Entrada:

Puntos obtenidos (puntos)

Tiempo jugado (tiempo)

Nivel alcanzado (nivel)

Proceso:

1. Declarar las constantes simbolicas BONO_TIEMPO = 0.10, BONO_NIVEL = 100, TIEMPO_MIN = 30.0, NIVEL_MAX = 10
2. Declarar las variables puntos, nivel como enteros, tiempo, bono_tiempo, bono_nivel, puntuacion_total, minimo como reales
3. Recibir por teclado los puntos obtenidos, tiempo jugado y nivel alcanzado
4. Verificar que (puntos >= 0) Y (nivel >= 1) Y (nivel <= NIVEL_MAX)
5. Si no se cumple mostrar mensaje de error y finalizar programa
6. Verificar (tiempo < TIEMPO_MIN)
7. Si se cumple calcular bono_tiempo = puntos * BONO_TIEMPO
8. Si no se cumple bono_tiempo = 0.0
9. Calcular bono_nivel = nivel * BONO_NIVEL
10. Calcular puntuacion_toal = puntos + bono_nivel + bono_tiempo
11. Asignar a minino el valor de puntos
12. Calcular el valor mínimo de todas las variables, utilizando una estructura secuencial comparando la variable minimo con el resto de variables y asignando el menor valor a minimo
13. Mostrar por terminal la puntuación total y el valor mínimo de todas las variables

Datos Salida:

Mensaje con la puntuación total

Mensaje con el valor minimo de todas las variables

Mensaje de error cuando no se cumpla una de las condiciones

Diagrama de flujo

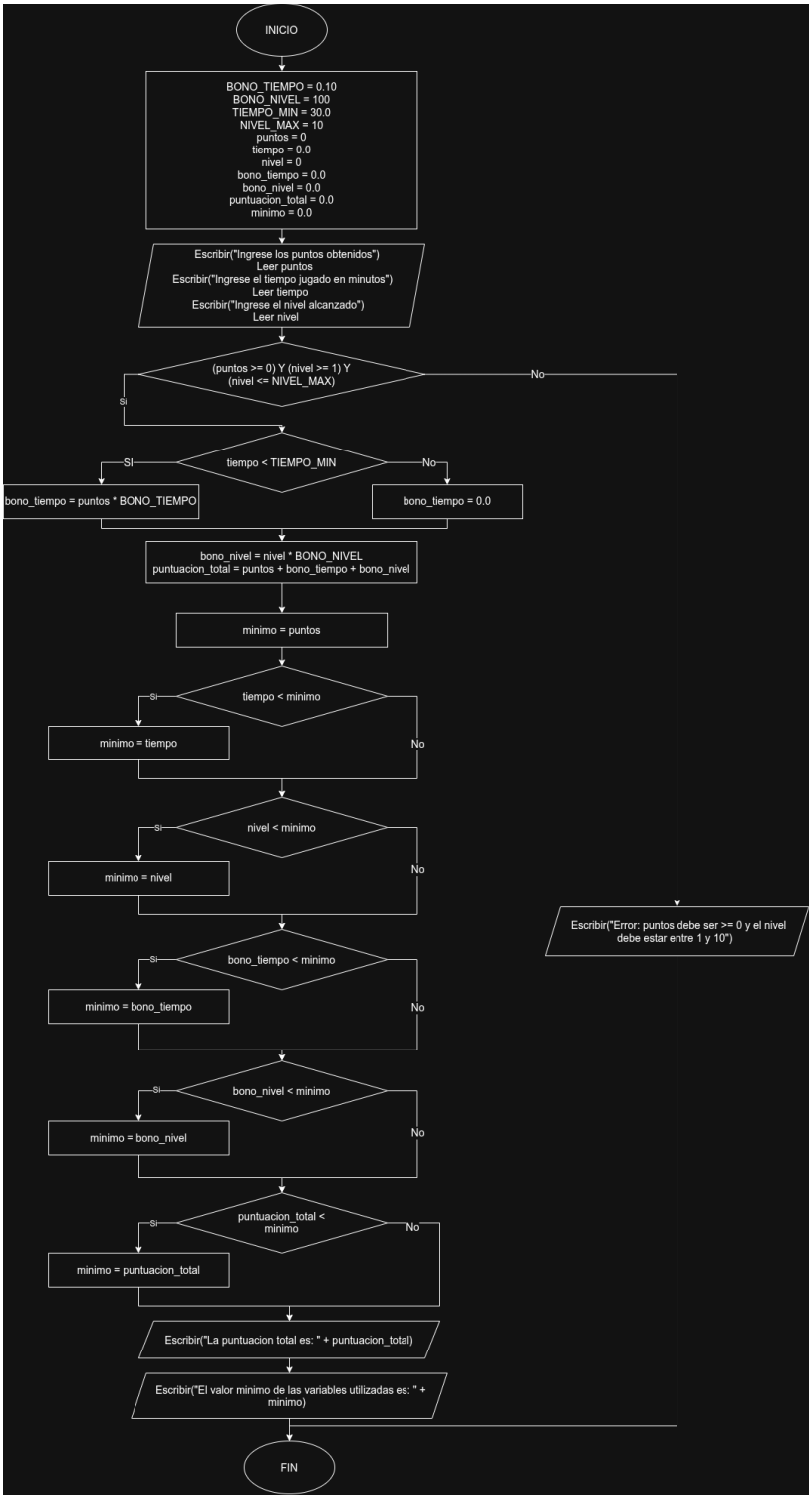


Fig.61. Diagrama de flujo del ejercicio 077

Pseudocódigo

```
Inicio

    BONO_TIEMPO = 0.10
    BONO_NIVEL = 100
    TIEMPO_MIN = 30.0
    NIVEL_MAX = 10

    puntos = 0
    tiempo = 0.0
    nivel = 0
    bono_tiempo = 0.0
    bono_nivel = 0.0
    puntuacion_total = 0.0
    minimo = 0.0

    escribir("Ingrese los puntos obtenidos")
    leer puntos

    escribir("Ingrese el tiempo jugado en minutos")
    leer tiempo

    escribir("Ingrese el nivel alcanzado")
    leer nivel

    si (puntos >= 0) Y (nivel >= 1) Y (nivel <= NIVEL_MAX) entonces

        si (tiempo < TIEMPO_MIN) entonces
            bono_tiempo = puntos * BONO_TIEMPO
        sino
```

```
            bono_tiempo = 0.0
        FinSi

        bono_nivel = nivel * BONO_NIVEL
        puntuacion_total = puntos + bono_tiempo + bono_nivel

        minimo = puntos
        si (tiempo < minimo) entonces
            minimo = tiempo
        FinSi
        si (nivel < minimo) entonces
            minimo = nivel
        FinSi
        si (bono_tiempo < minimo) entonces
            minimo = bono_tiempo
        FinSi
        si (bono_nivel < minimo) entonces
            minimo = bono_nivel
        FinSi
        si (puntuacion_total < minimo) entonces
            minimo = puntuacion_total
        FinSi

        escribir("La puntuación total es: " + puntuacion_total)
        escribir("El valor mínimo de las variables utilizadas es: " + minimo)

sino
        escribir("Error: puntos debe ser >= 0 y el nivel debe estar entre 1 y 10")
FinSi
```

Fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
puntos	150	La puntuación total es: 565.0 El valor mínimo de las variables utilizadas es: 4.0
tiempo	20.0	
nivel	4	
bono_tiempo	15.0	
bono_nivel	400.0	
puntuacion_total	565.0	
minimo	4.0	

Prueba 2

Variable	Valor	Resultado
puntos	100	La puntuación total es: 300.0 El valor mínimo de las variables utilizadas es: 0.0
tiempo	45.0	
nivel	2	
bono_tiempo	0.0	
bono_nivel	200.0	
puntuacion_total	300.0	
minimo	0.0	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{
    float BONO_TIEMPO = 0.10;
    int BONO_NIVEL = 100;
    float TIEMPO_MIN = 30.0;
    int NIVEL_MAX = 10;

    int puntos = 0;
    float tiempo = 0.0;
    int nivel = 0;
    float bono_tiempo = 0.0;
    float bono_nivel = 0.0;
    float puntuacion_total = 0.0;
    float minimo = 0.0;

    printf("Ingrese los puntos obtenidos\n");
    scanf("%d",&puntos);
    printf("Ingrese el tiempo jugado en minutos\n");
    scanf("%f",&tiempo);
    printf("Ingrese el nivel alcanzado\n");
    scanf("%d",&nivel);

    if (puntos >= 0 && nivel >= 1 && nivel < NIVEL_MAX)
    {
        if (tiempo < TIEMPO_MIN)
        {
            bono_tiempo = puntos * BONO_TIEMPO;
        }
        else
        {
            bono_tiempo = 0;
        }

        bono_nivel = nivel * BONO_NIVEL;
        puntuacion_total = puntos + bono_nivel + bono_tiempo;
        minimo = puntos;
        if ( tiempo< minimo)
        {
            minimo = tiempo;
        }
        if ( nivel < minimo)
        {
            minimo = nivel;
        }
        if ( bono_tiempo < minimo)
        {
            minimo = bono_tiempo;
        }
        if ( bono_nivel < minimo)
        {
            minimo = bono_nivel;
        }
        if ( puntuacion_total < minimo)
        {
            minimo = puntuacion_total;
        }


        printf("La puntuación total es %.2f\n",puntuacion_total);
        printf("El valor minimo de las variables utilizadas es %.2f\n",minimo);


    }
    else
    {
        printf("Error: puntos debe ser >= 0 y el nivel debe estar entre 1 y 10 \n");
    }



    return 0;
}

```

Fig.62. Código en C del ejercicio 077

4.7. Salida del programa

```
Ingrese los puntos obtenidos
150
Ingrese el tiempo jugado en minutos
20
Ingrese el nivel alcanzado
4
La puntuación total es 565.00
El valor minimo de las variables utilizadas es 4.00
```

Fig.63. Salida por terminal del programa

```
Ingrese los puntos obtenidos
100
Ingrese el tiempo jugado en minutos
45
Ingrese el nivel alcanzado
2
La puntuación total es 300.00
El valor minimo de las variables utilizadas es 0.00
```

Fig.64. Salida por terminal del programa

Conclusiones:

-El bono por tiempo condicional (tiempo < TIEMPO_MIN ? 1 : 0) muestra cómo una condición puede activar o desactivar un bonus en un sistema de puntuación

-La inicialización de todas las variables antes de usarlas es una buena práctica que evita valores basura en el cálculo de la puntuación total

EJ-078

En un taller mecanico, diseñe un algoritmo que valide el numero de una tarjeta de credito (cadena de 16 digitos) usando el algoritmo de Luhn. Use constantes simbolicas LONGITUD_TARJETA = 16, MODULO = 10. Use una variable entera suma inicializada con literal 0. Aplique el algoritmo: duplicar los digitos en posiciones pares, sumar los digitos de los resultados. Calcule la suma total con expresiones aritmeticas. Evalúe la expresion logica (suma % MODULO == 0). Asigne el resultado a tarjeta_valida. Muestre por pantalla la validacion. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

Análisis del problema

Datos Entrada

Numero de tarjeta de credito (tarjeta)

Proceso:

1. Declarar las constantes simbolicas LONGITUD_TARJETA = 16, MODULO = 10
2. Declarar las variables tarjeta como cadena, suma, digito, digito_doble, minimo, cont_operaciones como enteros, tarjeta_valida como booleano

3. Recibir por teclado el numero de la tarjeta de credito
4. Verificar que la longitud de la cadena sea igual a LONGITUD_CADENA
5. Si no se cumple mostrar mensaje de error y finalizar el programa
6. Aplicar el algoritmo de luhn para cada uno de los números de la tarjeta
7. En las posiciones impares extraer el dígito, calcular digito_doble = digito * 2; si digito_doble > 9 entonces digito_doble = digito_doble - 9; sumar digito_doble a suma
8. En las posiciones pares extraer el dígito y sumarlo directamente a suma
9. En cada operación de suma, incrementar el conteo de operaciones en 1
10. Evaluar la expresión relacional (suma % MODULO == 0) y asignar el resultado a tarjeta_valida
11. Evaluar la expresión (tarjeta_valida == true)
12. Si es verdadero mostrar mensaje “La tarjeta es valida”
13. Si es falso mostrar mensaje “La tarjeta no es valida”
14. Asignar a minimo el valor de suma
15. Calcular el valor minimo de las variables, comparando la variable minimo con el resto de variables y asignando el menor valor a esta misma
16. Mostrar por terminal el valor mínimo de las variables utilizadas

Datos Salida

Mensaje indicando si la tarjeta es valida o no

Mensaje con el valor mínimo de todas las variables

Mensaje de error si la tarjeta no tiene 17 digitos

Diagrama de flujo

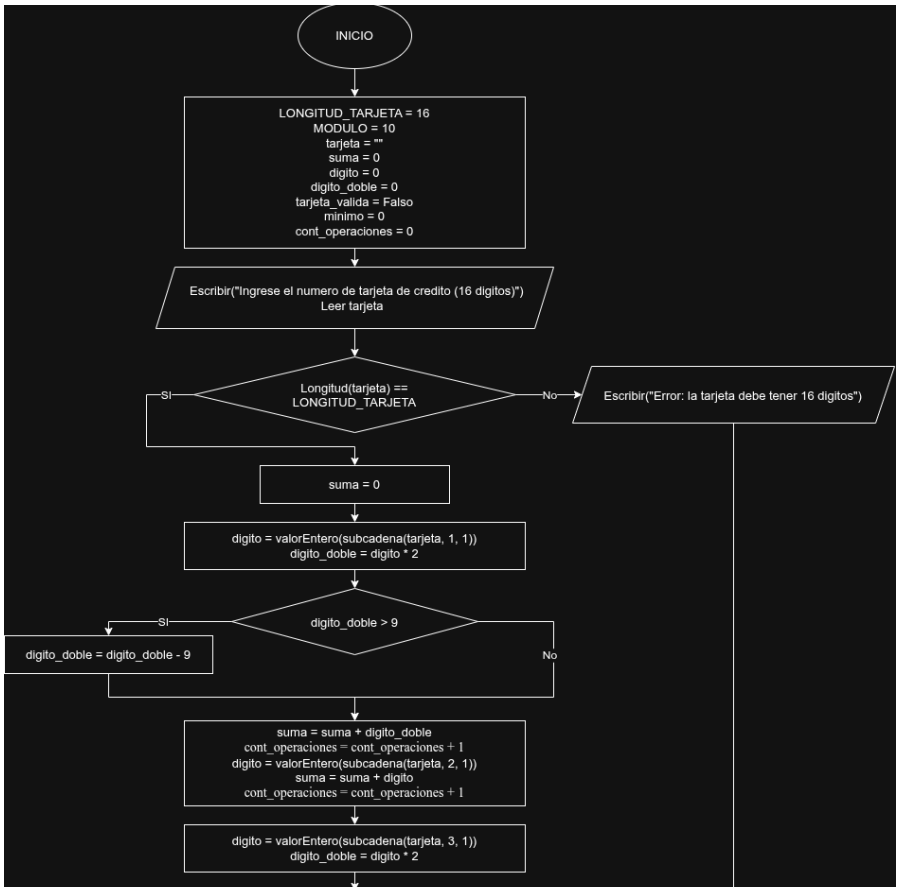


Fig.65. Diagrama de flujo del ejercicio 078

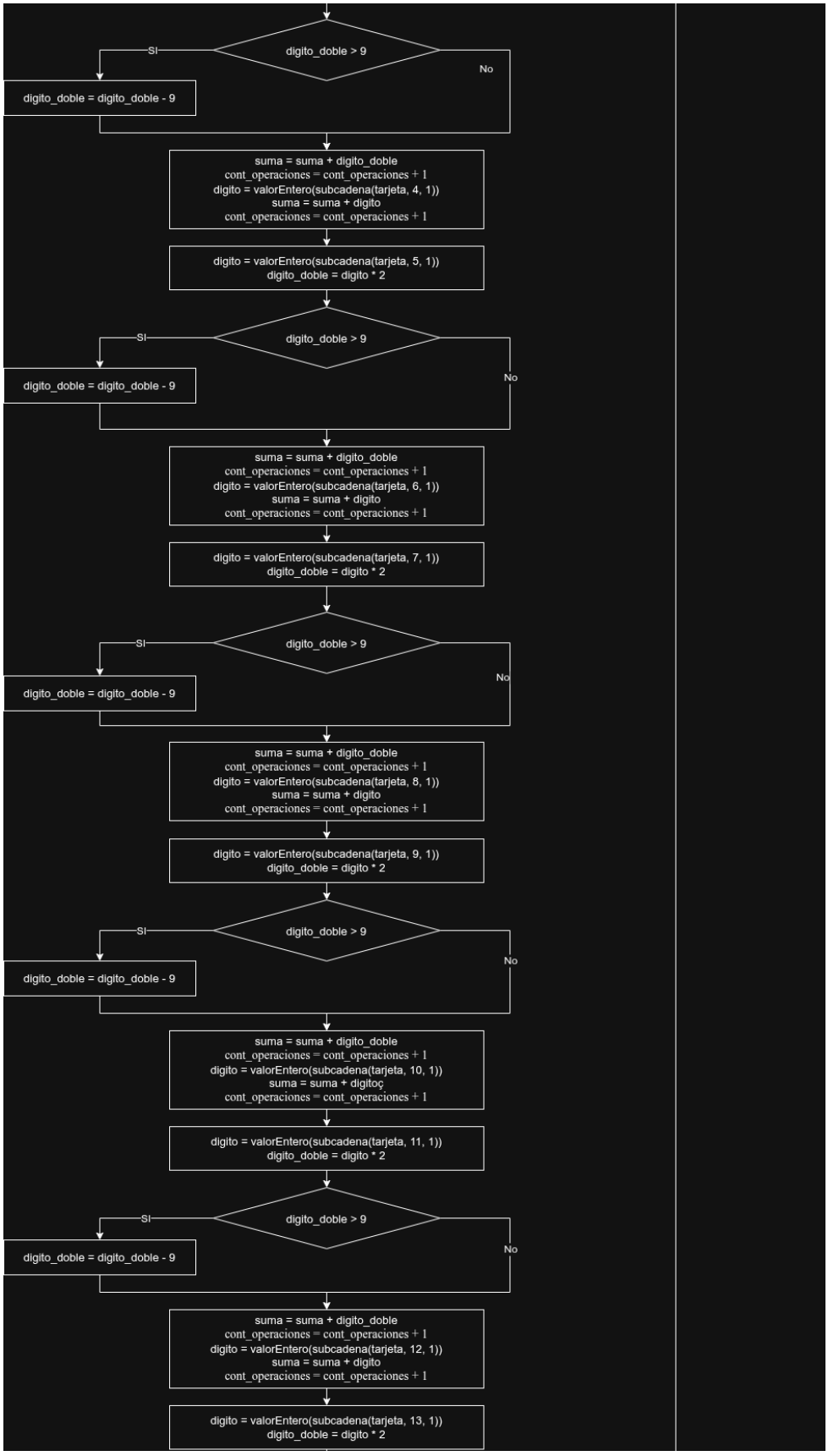


Fig.66. Diagrama de flujo del ejercicio 078

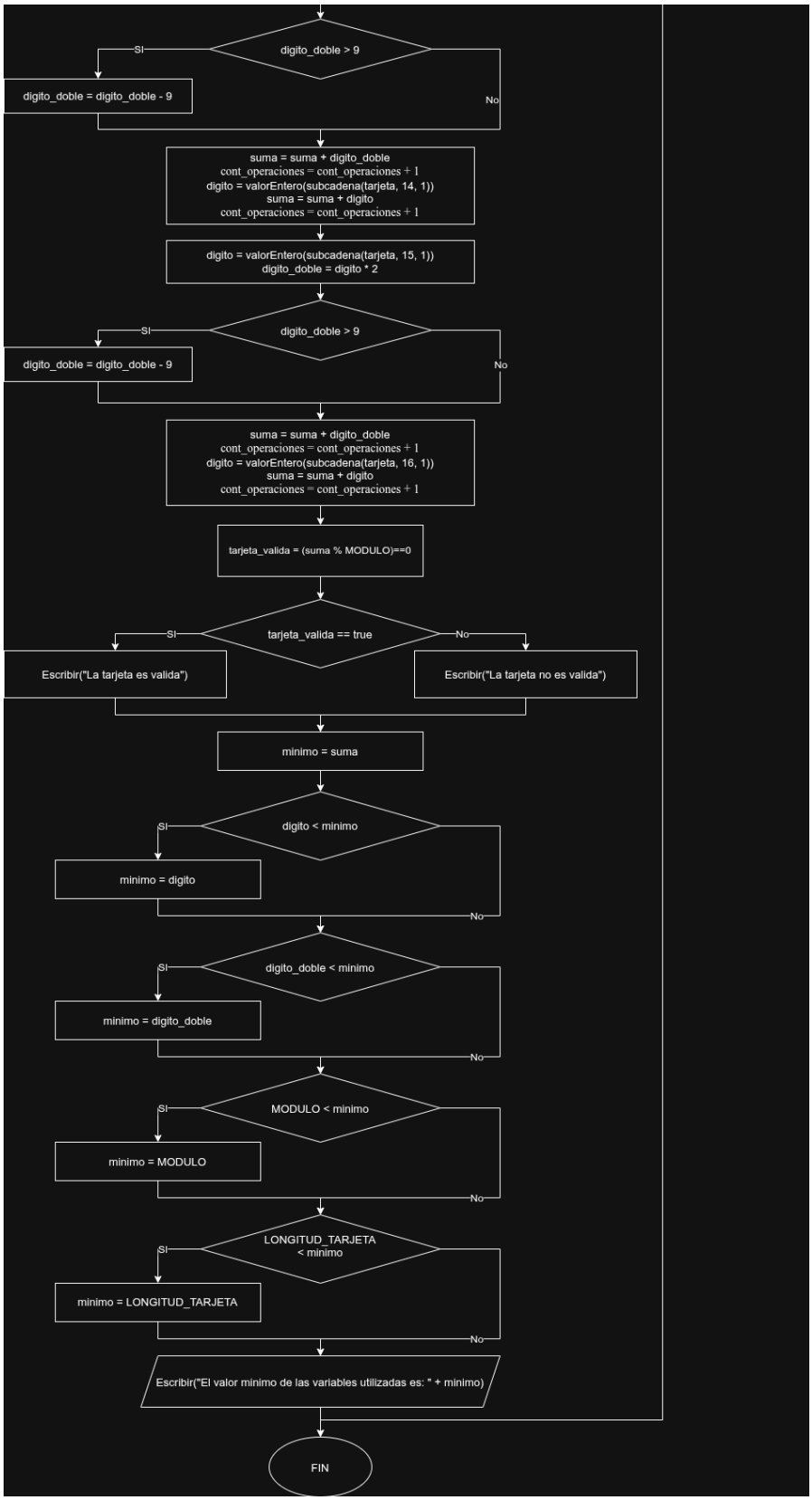


Fig.67. Diagrama de flujo del ejercicio 078

Pseudocódigo

```
Inicio
        LONGITUD_TARJETA = 16
        MODULO = 10

        tarjeta = ""
        suma = 0
        digito = 0
        digito_doble = 0
        tarjeta_valida = falso
        minimo = 0
        cont_operaciones = 0

        escribir("Ingrese el numero de tarjeta de credito (16 digitos)")
        leer tarjeta

        si longitud(tarjeta) == LONGITUD_TARJETA entonces
                suma = 0

                digito = valorEntero(subcadena(tarjeta,1,1))
                digito_doble = digito * 2
                si digito_doble > 9 entonces
                        digito_doble = digito_doble - 9
                FinSi
                suma = suma + digito_doble
                cont_operaciones = cont_operaciones + 1

                digito = valorEntero(subcadena(tarjeta,2,1))
                suma = suma + digito
                cont_operaciones = cont_operaciones + 1
```

```
digito = valorEntero(subcadena(tarjeta,3,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,4,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,5,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,6,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,7,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
```

```
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,8,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,9,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,10,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,11,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,12,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1
```

```
digito = valorEntero(subcadena(tarjeta,13,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,14,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,15,1))
digito_doble = digito * 2
si digito_doble > 9 entonces
        digito_doble = digito_doble - 9
FinSi
suma = suma + digito_doble
cont_operaciones = cont_operaciones + 1

digito = valorEntero(subcadena(tarjeta,16,1))
suma = suma + digito
cont_operaciones = cont_operaciones + 1

tarjeta_valida = (suma % MODULO) == 0
si (tarjeta_valida == true) entonces
        escribir("La tarjeta es valida")
sino
        escribir("La tarjeta no es valida")
```

```
            FinSi

            minimo = suma
            si digito < minimo entonces
                minimo = digito
                cont_operaciones = cont_operaciones + 1
            FinSi
            si digito_doble < minimo entonces
                minimo = digito_doble
                cont_operaciones = cont_operaciones + 1
            FinSi
            si MODULO < minimo entonces
                minimo = MODULO
                cont_operaciones = cont_operaciones + 1
            FinSi
            si LONGITUD_TARJETA < minimo entonces
                minimo = LONGITUD_TARJETA
                cont_operaciones = cont_operaciones + 1
            FinSi

            escribir("El valor minimo de las variables utilizadas es: " + minimo)
        sino
            escribir("Error: la tarjeta debe tener 16 digitos")
        FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
tarjeta	"1234567812345670"	La tarjeta es valida El valor minimo de las variables utilizadas es: 60
suma	60	
digito	0	
digito_doble	0	
tarjeta_valida	true	
minimo	60	

Prueba 2

Variable	Valor	Resultado
tarjeta	"1234567812345675"	La tarjeta no es valida El valor minimo de las variables utilizadas es: 5
suma	65	
digito	5	
digito_doble	0	
tarjeta_valida	false	
minimo	5	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

int main()
{

    int LONGITUD_TARJETA = 16;
    int MODULO = 10;

    char tarjeta[17];
    int suma = 0;
    int digito = 0;
    int digito_doble = 0;
    int minimo = 0;
    bool tarjeta_valida = false;
    int cont_operaciones = 0;

    printf("Ingrese el numero de tarjeta de credito (16 digitos)");
    fgets(tarjeta,sizeof(tarjeta),stdin);

    if (strlen(tarjeta) == LONGITUD_TARJETA)
    {
        suma = 0;


        digito = tarjeta[0] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[1] - '0';
        suma +=digito;
        cont_operaciones +=1;

        digito = tarjeta[2] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[3] - '0';
        suma +=digito;
        cont_operaciones +=1;

        digito = tarjeta[4] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[5] - '0';
        suma +=digito;
        digito = tarjeta[6] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;
```

Fig.68. Código en C del ejercicio 078

```
        digito = tarjeta[7] - '0';
        suma +=digito;
        digito = tarjeta[8] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[9] - '0';
        suma +=digito;
        digito = tarjeta[10] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[11] - '0';
        suma +=digito;
        digito = tarjeta[12] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[13] - '0';
        suma +=digito;
        cont_operaciones +=1;

        digito = tarjeta[14] - '0';
        digito_doble = digito * 2;
        if (digito_doble > 9)
        {
            digito_doble -= 9;
        }
        suma += digito_doble;
        cont_operaciones +=1;

        digito = tarjeta[15] - '0';
        suma +=digito;
        cont_operaciones +=1;

        tarjeta_valida = (suma % MODULO) == 0;

        if (tarjeta_valida == true)
        {
            printf("La tarjeta es valida\n");
        }
        else
        {
            printf("La tarjeta no es valida\n");
        }

        minimo = suma;
        if (digito < minimo)
        {
            minimo = digito;
            cont_operaciones +=1;
        }
        if (digito_doble < minimo)
        {
            minimo = digito_doble;
            cont_operaciones +=1;
        }
        if (MODULO < minimo)
        {
            minimo = MODULO;
            cont_operaciones +=1;
        }
        if (LONGITUD_TARJETA < minimo)
        {
            minimo = LONGITUD_TARJETA;
            cont_operaciones +=1;
        }
```

Fig.69. Código en C del ejercicio 078

```
156         printf("El valor minimo de las variables utilizadas es %d\n",minimo);
157
158
159
160
161     }
162     else
163     {
164         printf("Error: la tarjeta debe tener 16 digitos\n");
165     }
166
167
168     return 0;
169 }
170
```

Fig.70. Código en C del ejercicio 078

4.7. Salida del programa

```
➜ Trabajo_final git:(main) ✗ ./ejercicio078
Ingrese el numero de tarjeta de credito (16 digitos)
1234567812345670
La tarjeta es valida
El valor minimo de las variables utilizadas es 0
```

Fig.71. Salida por terminal del programa

```
➜ Trabajo_final git:(main) ✗ ./ejercicio078
Ingrese el numero de tarjeta de credito (16 digitos)
1234567812345675
La tarjeta no es valida
El valor minimo de las variables utilizadas es 5
```

Fig.72. Salida por terminal del programa

Conclusiones:

- El algoritmo de Luhn (duplicar dígitos, restar 9 si supera 9, sumar) es el método estándar de validación de números de tarjeta, demostrando su lógica paso a paso

-El uso de un counter de operaciones (cont_operaciones) muestra cómo una variable auxiliar puede llevar la cuenta de las veces que se ejecuta una operación

EJ-079

En una escuela, construya un algoritmo que lea el radio y la altura de un cilindro (dos reales). Use constantes simbolicas PI = 3.14159. Calcule: area_base = PI * radio * radio; area_lateral = 2 * PI * radio * altura; area_total = 2 * area_base + area_lateral; volumen = area_base * altura. Asigne los resultados a variables reales. Evalúe la expresion logica (radio > 0 AND altura > 0). Muestre por pantalla todas las medidas. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados.

Análisis del problema

Datos Entrada

Radio del cilindro (radio)

Altura del cilindro (altura)

Procesos:

1. Declarar la constante simbólica PI = 3.14159
2. Declarar las variables radio, altura, area_base, area_lateral, area_total, volumen y minimo como reales
3. Recibir por teclado el radio y altura del cilindro
4. Verificar la expresión lógica (radio > 0) Y (altura > 0)
5. Si no se cumple mostrar un mensaje de error y finalizar el programa
6. Calcular el área de la base con area_base = PI * radio * radio.
7. Calcular el área lateral con area_lateral = 2 * PI * radio * altura.
8. Calcular el área total con area_total = 2 * area_base + area_lateral.
9. Calcular el volumen con volumen = area_base * altura
10. Asignar a minimo el valor de radio
11. Calcular el valor minimo de las variables utilizadas comparando de forma secuencial el valor de minimo con el del resto de las variables y asignar el menor a minimo
12. Mostrar por terminal el valor del radio, de la altura, del área de la base, del área lateral, del área total y del volumen
13. Mostrar por terminal el valor mínimo de todas las variables utilizadas

Datos Salida

Mensajes con los valores de radio, altura, area_base, area_lateral, area_total, y volumen

Mensaje con el valor mínimo de todas las variables utilizadas

Mensaje de error cuando el radio o la altura son menores que 0

14.

Diagrama de flujo

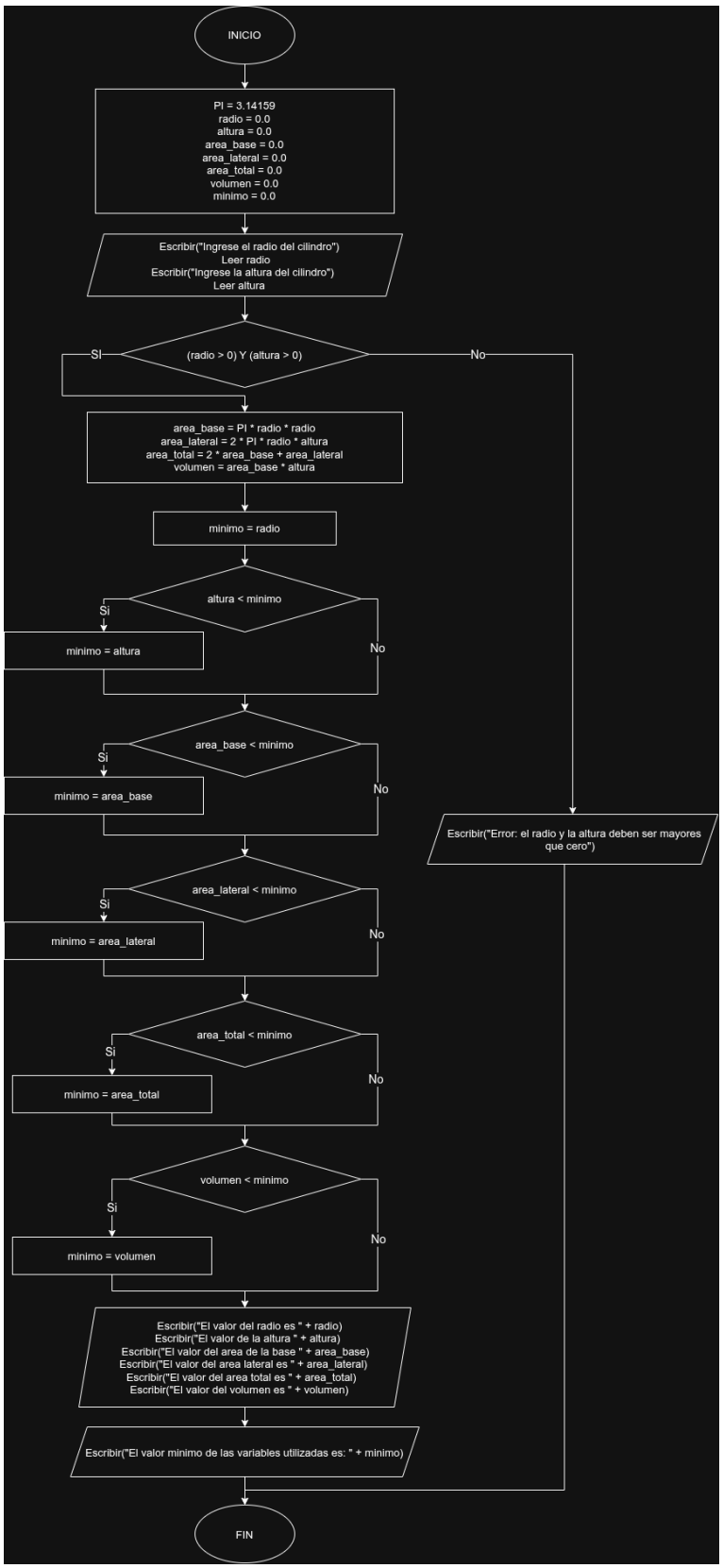


Fig.73. Diagrama de flujo del ejercicio 079

Pseudocódigo

```
Inicio
        PI = 3.14159

        radio = 0.0
        altura = 0.0
        area_base = 0.0
        area_lateral = 0.0
        area_total = 0.0
        volumen = 0.0
        minimo = 0.0

        escribir("Ingrese el radio del cilindro")
        leer radio
        escribir("Ingrese la altura del cilindro")
        leer altura

        si (radio > 0) Y (altura > 0) entonces
                area_base = PI * radio * radio
                area_lateral = 2 * PI * radio * altura
                area_total = 2 * area_base + area_lateral
                volumen = area_base * altura
                minimo = radio

                si altura < minimo entonces
                        minimo = altura
                FinSi

                si area_base < minimo entonces
                        minimo = area_base
```

```
            FinSi

            si area_lateral < minimo entonces
                minimo = area_lateral
            FinSi

            si area_total < minimo entonces
                minimo = area_total
            FinSi

            si volumen < minimo entonces
                minimo = volumen
            FinSi

            escribir("El valor del radio es " + radio)
            escribir("El valor de la altura " + altura)
            escribir("El valor del area de la base " + area_base)
            escribir("El valor del area lateral es " + area_lateral)
            escribir("El valor del area total es " + area_total)
            escribir("El valor del volumen es " + volumen)
            escribir("El valor minimo de las variables utilizadas es: " + minimo)
        sino
            escribir("Error: el radio y la altura deben ser mayores que cero")
        FinSi
Fin
```

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
radio	3.0	El valor del radio es 3.0 El valor de la altura 5.0 El area de la base 28.2743 El area lateral es 94.2477 El area total es 150.7963 El volumen es 141.3715 El valor minimo de las variables utilizadas es: 3.0
altura	5.0	
area_base	28.2743	
area_lateral	94.2477	
area_total	150.7963	
volumen	141.3715	
minimo	3.0	

Prueba 2

Variable	Valor	Resultado
radio	2.0	El valor del radio es 2.0 El valor de la altura 4.0 El area de la base 12.5664 El area lateral es 50.2654 El area total es 75.3982 El volumen es 50.2654 El valor minimo de las variables utilizadas es: 2.0
altura	4.0	
area_base	12.5664	
area_lateral	50.2654	
area_total	75.3982	
volumen	50.2654	
minimo	2.0	

Codificación en lenguaje C

```c
#include <stdio.h>

int main()
{
    float PI = 3.14159;
    float radio = 0.0;
    float altura = 0.0;
    float area_base = 0.0;
    float area_lateral = 0.0;
    float area_total = 0.0;
    float volumen = 0.0;
    float minimo = 0.0;

    printf("Ingrese el radio del cilindro\n");
    scanf("%f",&radio);
    printf("Ingrese la altura del cilindro\n");
    scanf("%f",&altura);

    if (radio > 0 && altura > 0)
    {
        area_base = PI * radio * radio;
        area_lateral = 2  * PI * radio * altura;
        area_total = 2 * area_base + area_lateral;
        volumen  =area_base * altura;

        minimo = radio;

        if (altura < minimo)
        {
            minimo  = altura;
        }
        if (area_base < minimo)
        {
            minimo  = area_base;
        }
        if (area_lateral < minimo)
        {
            minimo  = area_lateral;
        }
        if (area_total < minimo)
        {
            minimo  = area_total;
        }

        if (volumen < minimo)
        {
            minimo  = volumen;
        }

        printf("EL valor del radio es %.2f\n",radio);
        printf("EL valor de la altura es %.2f\n",altura);
        printf("EL valor del area de la base %.2f\n",area_base);
        printf("EL valor del area lateral es %.2f\n",area_lateral);
        printf("EL valor del area total es %.2f\n",area_total);
        printf("EL valor del volumen es %.2f\n",volumen);
        printf("EL valor minimo de las variables itilizadas es %.2f\n",minimo);

    }
    else
    {
        printf("Error: el radio y la altura deben ser mayores que cero\n");
    }



    return 0;
}

```

Fig.74. Código en C del ejercicio 079

4.7. Salida del programa

```
Ingrese el radio del cilindro
3
Ingrese la altura del cilindro
5
EL valor del radio es 3.00
EL valor de la altura es 5.00
EL valor del area de la base 28.27
EL valor del area lateral es 94.25
EL valor del area total es 150.80
EL valor del volumen es 141.37
EL valor minimo de las variables itilizadas es 3.00
```

Fig.75. Salida por terminal del programa

```
Ingrese el radio del cilindro
2
Ingrese la altura del cilindro
4
EL valor del radio es 2.00
EL valor de la altura es 4.00
EL valor del area de la base 12.57
EL valor del area lateral es 50.27
EL valor del area total es 75.40
EL valor del volumen es 50.27
EL valor minimo de las variables itilizadas es 2.00
```

Fig.76. Salida por terminal del programa

Conclusiones:

-El cálculo del valor mínimo de todas las variables (radio, altura, áreas, volumen) demuestra que la técnica de comparación secuencial sirve para cualquier conjunto de medidas

- Las fórmulas del área base, lateral, total y volumen muestran la aplicación de geometría tridimensional con la constante PI

EJ-080

En una libreria, cree un algoritmo que simule un sistema de votacion. Lea tres votos (enteros para candidatos A, B, C). Use constantes simbolicas VOTOS_MIN = 0, MAYORIA_ABSOLUTA = 0.5. Calcule el total de votos con (votos_a + votos_b + votos_c). Use expresiones relacionales y logicas para determinar el ganador. Evalúe las expresiones logicas: (votos_a > votos_b AND votos_a > votos_c), (votos_b > votos_a AND votos_b > votos_c), etc. Asigne el ganador a una variable cadena. Muestre por pantalla los resultados. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de que las expresiones logicas esten bien parentizadas.

Análisis del problema

Datos Entrada

Votos del candidato A (votos_a)

Votos del candidato B (votos_b)

Votos del candidato C (votos_c)

Procesos:

1. Declarar las constantes simbólicas VOTOS_MIN = 0 y MAYORIA_ABSOLUTA = 0.5
2. Declarar las variables votos_a, votos_b, votos_c, total_votos y minimo como enteras, ganador como tipo cadena
3. Recibir los votos para los candidatos A, B y C
4. Validar que (votos_a >= VOTOS_MIN) Y (votos_b >= VOTOS_MIN) Y (votos_c >= VOTOS_MIN). Si no se cumple, mostrar un mensaje de error y finalizar el programa
5. Calcular el total de votos con total_votos = votos_a + votos_b + votos_c
6. Evaluar las expresiones relacionales (total_votos == 0), (votos_a > votos_b) Y (votos_a > votos_c), (votos_b > votos_a) Y (votos_b > votos_c), (votos_c > votos_a) Y (votos_c > votos_b) , para determinar si no se introdujo ningún voto, cual de los 3 gano o en caso contrario señalar empate
7. Mostrar por terminal la cantidad de votos para cada candidato, el total de los votos y el candidato ganador
8. Calcular el valor mínimo de todas las variables inicializando minimo con votos_a, comparar con los valores del resto de variables y asigna a minimo el menor valor en cada caso
9. Mostrar por terminal el valor mínimo de las variables usadas

Datos Salida

Mensaje con la cantidad de votos a cada candidato

Mensaje con los votos totales

Mensaje con el ganador de la votación

Mensaje con el valor mínimo de las variables utilizadas

Mensaje de error si algún voto es negativo o si no se registraron votos

Diagrama de flujo

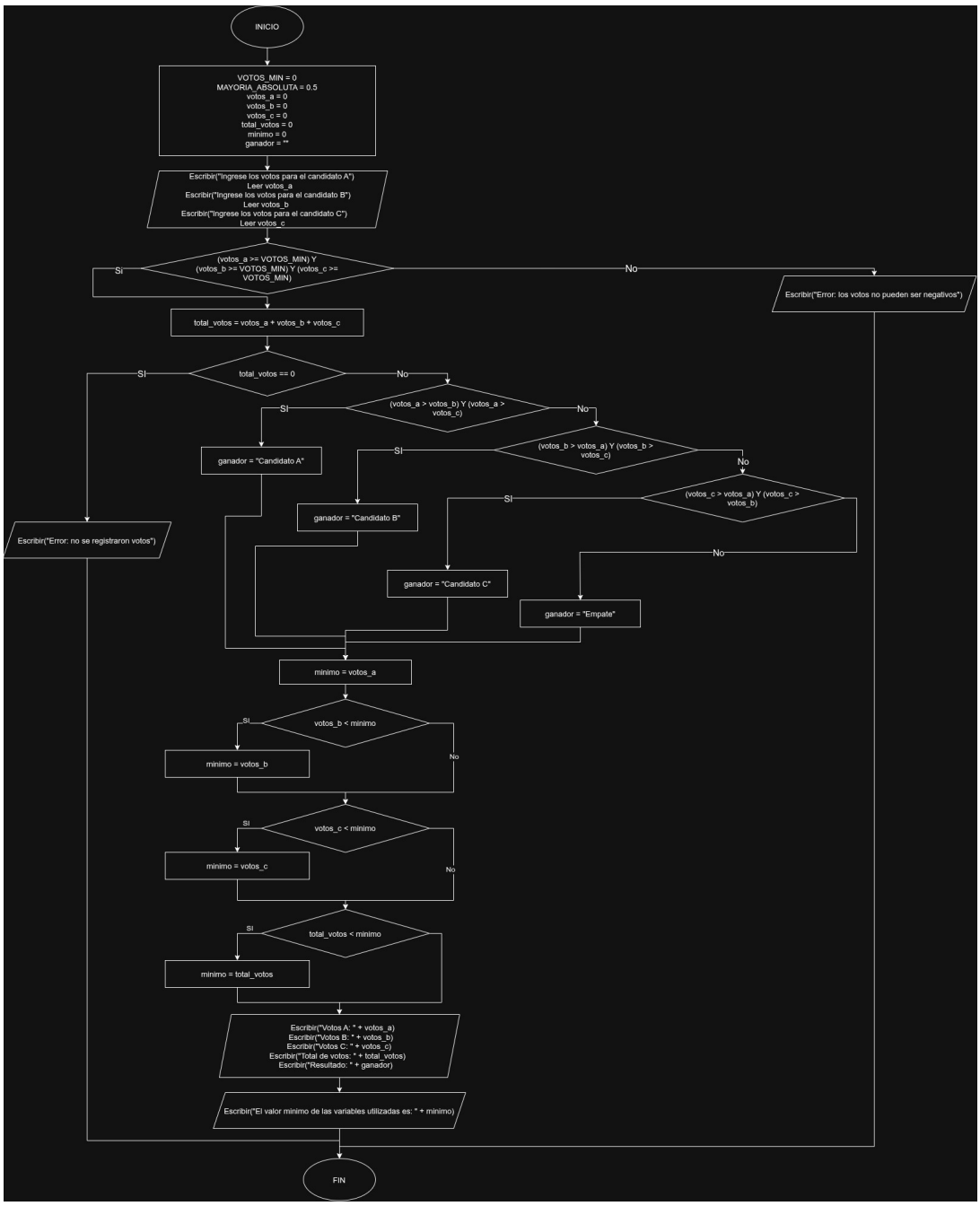


Fig.77. Diagrama de flujo del ejercicio 080

Pseudocódigo

```
Inicio

    VOTOS_MIN = 0
    MAYORIA_ABSOLUTA = 0.5

    votos_a = 0
    votos_b = 0
    votos_c = 0
    total_votos = 0
    minimo = 0
    ganador = ""

    escribir("Ingrese los votos para el candidato A")
    leer votos_a
    escribir("Ingrese los votos para el candidato B")
    leer votos_b
    escribir("Ingrese los votos para el candidato C")
    leer votos_c

    si (votos_a >= VOTOS_MIN) Y (votos_b >= VOTOS_MIN) Y (votos_c >= VOTOS_MIN) entonces

        total_votos = votos_a + votos_b + votos_c

        si (total_votos == 0) entonces
            escribir("Error: no se registraron votos")
            sino si ((votos_a > votos_b) Y (votos_a > votos_c)) entonces
                ganador = "Candidato A"
            sino si ((votos_b > votos_a) Y (votos_b > votos_c)) entonces
```

```
                ganador = "Candidato B"
            sino si ((votos_c > votos_a) Y (votos_c > votos_b)) entonces
                ganador = "Candidato C"
            sino
                ganador = "Empate"
            FinSi

            escribir("Votos A: " + votos_a)
            escribir("Votos B: " + votos_b)
            escribir("Votos C: " + votos_c)
            escribir("Total de votos: " + total_votos)
            escribir("Resultado: " + ganador)

            minimo = votos_a
            si (votos_b < minimo) entonces
                minimo = votos_b
            FinSi
            si (votos_c < minimo) entonces
                minimo = votos_c
            FinSi
            si (total_votos < minimo) entonces
                minimo = total_votos
            FinSi

            escribir("El valor mínimo de las variables utilizadas es: " + minimo)

        FinSi

sino
        escribir("Error: los votos no pueden ser negativos")
```

FinSi

Fin

Prueba de escritorio

Prueba 1

Variable	Valor	Resultado
votos_a	45	Votos A: 45 Votos B: 30 Votos C: 25 Total de votos: 100 Resultado: Candidato A El valor mínimo de las variables utilizadas es: 25
votos_b	30	
votos_c	25	
total_votos	100	
minimo	25	
ganador	"Candidato A"	

Prueba 2

Variable	Valor	Resultado
votos_a	20	Votos A: 20 Votos B: 55 Votos C: 25 Total de votos: 100 Resultado: Candidato B El valor mínimo de las variables utilizadas es: 20
votos_b	55	
votos_c	25	
total_votos	100	
minimo	20	
ganador	"Candidato B"	

Codificación en lenguaje C

```c
#include <stdio.h>
#include <string.h>

int main()
{
    int VOTOS_MIN = 0;
    float MAYORIA_ABSOLUTA = 0.5;

    int votos_a = 0;
    int votos_b = 0;
    int votos_c = 0;
    int total_votos = 0;
    int minimo = 0;
    char ganador[30];

    printf("Ingrese los votos por el candidato A\n");
    scanf("%d",&votos_a);
    printf("Ingrese los votos por el candidato B\n");
    scanf("%d",&votos_b);
    printf("Ingrese los votos por el candidato C\n");
    scanf("%d",&votos_c);

    if (votos_a >= VOTOS_MIN && votos_b >= VOTOS_MIN && votos_c >= VOTOS_MIN)
    {
        total_votos = votos_a + votos_b + votos_c;

        if (total_votos == 0)
        {
            printf("No se registraron los votos\n");
        }
        else if (votos_a > votos_b && votos_a > votos_c)
        {
            strcpy(ganador,"Candidato A");
        }
        else if (votos_b > votos_a && votos_b > votos_c)
        {
            strcpy(ganador,"Candidato B");
        }
        else if (votos_c > votos_b && votos_c > votos_b)
        {
            strcpy(ganador,"Candidato C");
        }
        else
        {
            strcpy(ganador,"Empate");
        }

        minimo = votos_a;
        if (votos_b < minimo)
        {
            minimo = votos_b;
        }
        if (votos_c < minimo)
        {
            minimo = votos_c;
        }
        if (total_votos < minimo)
        {
            minimo = total_votos;
        }

        printf("El numero de votos por A es %d\n", votos_a);
        printf("El numero de votos por B es %d\n", votos_b);
        printf("El numero de votos por C es %d\n", votos_c);
        printf("El numero total de votos es %d\n", total_votos);
        printf("Resultado: %s\n", ganador);
        printf("El valor minimo de las variables utilizadas es %d\n",minimo);
    }

    else
    {
        printf("Error: los votos no pueden ser negativos\n");
    }



    return 0;
}

```

Fig.78. Código en C del ejercicio 080

4.7. Salida del programa

```
Ingrese los votos por el candidato A
45
Ingrese los votos por el candidato B
30
Ingrese los votos por el candidato C
25
El numero de votos por A es 45
El numero de votos por B es 30
El numero de votos por C es 25
El numero total de votos es 100
Resultado: Candidato A
El valor minimo de las variables utilizadas es 25
```

Fig.79. Salida por terminal del programa

```
Ingrese los votos por el candidato A
20
Ingrese los votos por el candidato B
55
Ingrese los votos por el candidato C
25
El numero de votos por A es 20
El numero de votos por B es 55
El numero de votos por C es 25
El numero total de votos es 100
Resultado: Candidato B
El valor minimo de las variables utilizadas es 20
```

Fig.80. Salida por terminal del programa

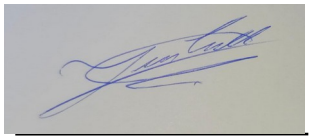
Conclusiones:

-Las expresiones lógicas bien parentizadas (votos_a > votos_b) AND (votos_a > votos_c) permiten determinar el ganador de forma inequívoca

- El manejo de empates y del caso total_votos == 0 muestra la importancia de considerar todos los escenarios posibles en un sistema de toma de decisiones

📄 Declaración de autoría

Declaro que el presente trabajo es de mi autoría, que no se utilizó inteligencia artificial (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma Estudiante

Docente